Facultatea de Matematică și Informatică

Universitatea București

Document de Analiză și Proiectare pentru Aplicația Software

Platformă pentru Analiza Sentimentului în Știri Financiare

Dinu Bogdan-Marius

Adresa de email – bogdan-marius.dinu@s.unibuc.ro

Data: 11.05.2025

Contents

1. Introducere 3

1.1 Scopul documentului 3

1.2 Domeniul de aplicare 4

1.3 Definiții, acronime și abrevieri 4

1.4 Referințe 5

1.5 Prezentare generală 5

2. Analiza cerințelor 7

2.1 Cerințe funcționale 7

2.2 Cerințe non-funcționale 8

2.3 Cazuri de utilizare (Use Cases) 9

2.4 Constrângeri 10

2.5 Riscuri 11

3. Arhitectura și proiectarea sistemului 13

3.1 Arhitectura generală 13

3.2 Modelul de date 14

3.3 Diagrame UML (Unified Modelling Language) 14

3.4 Tehnologii și stack de dezvoltare 16

3.5 Interfața utilizatorului (UI/UX) 16

4. Implementare 18

4.1 Structura codului 18

4.2 Module și componente 19

4.3 Fluxul de date 20

5. Testare 22

5.1 Strategia de testare 22

5.2 Tipuri de teste 23

5.3 Plan de testare 24

6. Deployment și mentenanță 26

6.1 Strategie de deployment 26

6.2 Administrarea și monitorizarea 27
6.3 Plan de mentenanță 28
7. Concluzii..... 30

1. Introducere

1.1 Scopul documentului

Explicarea obiectivelor documentului de analiză și proiectare.

Acest document are scopul de a defini cerințele, arhitectura și proiectarea unei aplicații software, oferind o bază solidă pentru implementare și testare. El servește drept ghid pentru echipa de dezvoltare și părțile interesate, asigurând o înțelegere clară a funcționalităților, cerințelor non-funcționale și constrângerilor tehnice. De asemenea, documentul detaliază procesul de testare și mentenanță pentru a garanta calitatea și securitatea aplicației.

1.2 Domeniul de aplicare

Descrierea domeniului aplicației și utilizatorilor vizați.

Aplicația software, denumită neoficial de mine, Market Sentiment Dashboard & AI Analyzer este destinată monitorizării și analizei sentimentului pieței financiare, cu focus pe piața de acțiuni din SUA.

Sistemul agregă știri financiare din multiple surse online(RSS feeds, NewsAPI), preia valoarea indicelui de volatilitate VIX și utilizează un model de inteligență artificială(DeepSeek, deployed prin Azure AI Services/AI Foundry) pentru a genera un sumar al sentimentului pieței și un indice numeric calculat estimativ de Fear & Greed.

Datele procesate(istoricul F&G, VIX, sumar AI) sunt stocate într-o bază de date PostGreSQL și prezentate utilizatorului printr-o interfață web interactivă(dashboard) construită cu Flask.

Utilizatorii vizați includ investitori individuali și orice persoană interesată de o perspectivă rapidă și inteligentă asupra stării pieței. Aplicația oferă și funcționalități de export de date(PDF, CSV) și control manual al actualizării datelor.

1.3 Definiții, acronime și abrevieri

Lista termenilor specifici utilizați în document.

AI: Artificial Intelligence (Inteligență Artificială) – Folosită pentru analiza de sentiment a știrilor

API: Application Programming Interface – Interfață prin care diferite componente software comunică (ex. NewsAPI, Azure AI API).

Azure AI Services: Platformă cloud Microsoft pentru servicii de inteligență articială; folosită pentru a hosta și interoga modelul DeepSeek

CDN: Content Delivery Network – Rețea de servere pentru livrarea rapidă a conținutului static(ex. Bilbiotecile JavaScript)

CRUD: Create, Read, Update, Delete – Operațiuni de bază de date(relevant pentru interacțiunea cu DB).

CSS: Cascading Style Sheets – Tehnologie pentru stilizarea paginilor web.

CSV: Comma-Separated Values – Format de fișier text pentru date tabelare.

DB: DataBase(Bază de Date) – În acest proiect, PostgreSQL.

DeepSeek: Numele modelului lingvistic(LLM) folosit pentru analiza AI.

ERD: Entitiy-Relation Diagram – Diagramă pentru modelarea structurii bazei de date.

F&G Index: Fear & Greed Index – Indicele "Frică și Lăcomie", o măsură a sentimentului investitorilor. Proiectul generează o versiune proprie.

Flask: Micro-framework web Python folosit pentru backend-ul aplicației.

Git: Sistem de control al versiunilor.

GitHub: Platformă de găzduire pentru repository-uri Git.

HTML: HyperText Markup Language – Limbajul de structurare a paginilor web.

HTTP: Hypertext Transfer Protocol – Protocolul de comunicare pe web.

JSON: JavaScript Object Notation – Format de schimb de date ușor de citit.

LLM: Large Language Model – Tip de model AI (ex: DeepSeek).

NLP: Natural Language Processing – Procesarea limbajului natural, domeniul AI relevant.

PDF: Portable Document Format – Format de fișier pentru documente.

PostgreSQL: Sistem de management al bazelor de date relaționale open-source.

psycopg2: Adaptor Python pentru PostgreSQL.

Pytest: Framework de testare pentru Python.

Python: Limbajul de programare principal folosit pentru backend și scripturi.

REST(ful): Representational State Transfer – Stil arhitectural pentru API-uri web.

RSS: Really Simple Syndication – Format pentru distribuirea de conținut web actualizat frecvent.

SDK: Software Development Kit – Set de unelte pentru dezvoltare (ex: azure-ai-inference).

SQL: Structured Query Language – Limbaj pentru interogarea bazelor de date.

UI: User Interface (Interfață Utilizator).

UX: User Experience (Experiență Utilizator).

VENV: Virtual Environment (Mediu Virtual Python).

VIX: Volatility Index – Indicele de volatilitate al pieței S&P 500.

yfinance: Bibliotecă Python pentru accesarea datelor financiare de pe Yahoo Finance.

1.4 Referințe

Listează documente, standarde și surse utilizate pentru elaborarea documentului.

Această secțiune conține lista documentelor, standardelor și surselor utilizate în procesul de analiză și proiectare a aplicației "Market Sentiment Dashboard & AI Analyzer". Referințele selectate au ghidat dezvoltarea soluției, asigurând alinierea la bune practici din industrie și o fundamentare teoretică și tehnică solidă.

Documente și Standarde

- ISO/IEC 25010:2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models: Acest standard a fost consultat pentru a înțelege caracteristicile de calitate relevante pentru un produs software, cum ar fi funcționalitatea, fiabilitatea, utilizabilitatea, eficiența performanței, mentenabilitatea și portabilitatea, principii care au ghidat dezvoltarea aplicației.[1][2][3][4]
- OWASP Top 10 Web Application Security Risks: Deși nu este o implementare cu focus extensiv pe securitate avansată, principiile OWASP Top 10 (de ex., prevenirea injecțiilor, managementul corect al autentificării și sesiunilor) au fost luate în considerare la nivel conceptual pentru dezvoltarea componentei web și gestionarea datelor sensibile (chei API).[5][6][7][8]
- UML 2.5 Specification – Object Management Group (OMG): Standardul a fost utilizat ca referință pentru crearea diagramelor UML (cazuri de utilizare, diagrame de secvență, diagrame de activitate) folosite în acest document pentru a modela și vizualiza arhitectura și funcționalitatea sistemului.[9][10][11][12][13]

- RFC 822 / RFC 1123 (Standarde pentru formatul datelor în email și HTTP): Relevante indirect prin utilizarea formatelor de dată standard în fluxurile RSS și API-uri, pentru care funcțiile de parsare a timestamp-urilor au fost dezvoltate.

Surse Web și Bibliografie

- Python Software Foundation, "Python 3.13 Documentation," python.org. https://docs.python.org/3/ (accesat 11 mai, 2025).
- Pallets Team, "Flask Documentation," palletsprojects.com. https://flask.palletsprojects.com/ (accesat 11 mai, 2025).
- PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," postgresql.org. https://www.postgresql.org/docs/16/index.html (accesat 11 mai, 2025).
- Microsoft Corporation, "Azure AI Services Documentation (Inference SDK)," microsoft.com. https://learn.microsoft.com/en-us/azure/ai-services/ (accesat 11 mai, 2025).
- NewsAPI, "NewsAPI Documentation," newsapi.org. https://newsapi.org/docs (accesat 11 mai, 2025).
- R. Aroussi, "yfinance Library Documentation," github.com. https://github.com/ranaroussi/yfinance (accesat 11 mai, 2025).
- Feedparser Contributors, "Feedparser Library Documentation," readthedocs.io. https://feedparser.readthedocs.io/ (accesat 11 mai, 2025).
- D. Bader, "Schedule Library Documentation," readthedocs.io. https://schedule.readthedocs.io/ (accesat 11 mai, 2025).
- Chart.js Contributors, "Chart.js Documentation," chartjs.org. https://www.chartjs.org/docs/latest/ (accesat 11 mai, 2025).
- B. Jakubowski, "Gauge.js Documentation," github.io. http://bernii.github.io/gauge.js/ (accesat 11 mai, 2025).
- N. von Hertzen, "html2canvas Documentation," hertzen.com. https://html2canvas.hertzen.com/ (accesat 11 mai, 2025).
- jsPDF Contributors, "jsPDF Documentation," githack.com. http://raw.githack.com/MrRio/jsPDF/master/docs/ (accesat 11 mai, 2025).
- Pytest Contributors, "Pytest Documentation," pytest.org. https://docs.pytest.org/ (accesat 11 mai, 2025).
- K. Reitz, "Requests Library Documentation," readthedocs.io. https://requests.readthedocs.io/ (accesat 11 mai, 2025).
- L. Richardson, "Beautiful Soup Documentation," crummy.com. https://www.crummy.com/software/BeautifulSoup/bs4/doc/ (accesat 11 mai, 2025).

Articole și Lucrări Academice Relevante

- **Niță, M., & Pochea, M. M. (2020). Time-varying dependence in European equity markets: A contagion and investor sentiment driven analysis. Economic Modelling, 86, 133-147.[14]**
- **Kearney, C., & Liu, S. (2014). Textual sentiment in finance: A survey of methods and models. International Review of Financial Analysis, 33, 171-190.**

- **Phillips, P. C. B., Shi, S., & Yu, J. (2015). Testing for multiple bubbles: Historical episodes of exuberance and collapse in the S&P 500. International Economic Review, 56(4), 1043-1078. 15]**

1.5 Prezentare generală

Descriere sumara a conținutului documentului.

Prezentul document constituie o analiză exhaustivă și o specificație de proiectare pentru aplicația software "Market Sentiment Dashboard & AI Analyzer". Structurat în capitole distincte, documentul urmărește un parcurs logic de la definirea conceptuală a proiectului până la considerații privind implementarea, testarea și mentenanța acestuia.

Capitolul introductiv stabilește scopul documentului, delimitează domeniul de aplicare al aplicației și clarifică terminologia specifică, acronimele și abrevierile utilizate. De asemenea, sunt enumerate referințele bibliografice și standardele care au fundamentat elaborarea proiectului.

Următorul capitol, dedicat analizei cerințelor, detaliază atât cerințele funcționale – specificând capabilitățile pe care sistemul trebuie să le ofere utilizatorului – cât și cerințele non-funcționale, care definesc atributele calitative ale sistemului, precum performanța, securitatea și utilizabilitatea. Această secțiune include și o prezentare a cazurilor de utilizare (Use Cases) principale, ilustrând interacțiunile cheie cu sistemul, alături de constrângerile tehnice, legale și operaționale identificate, și o analiză a riscurilor potențiale.

Capitolul subsecvent se concentrează pe arhitectura și proiectarea sistemului. Aici este prezentată arhitectura generală a aplicației, evidențiind componentele software majore și interacțiunile dintre acestea. Se detaliază modelul de date utilizat, inclusiv structura bazei de date, și sunt incluse diagrame UML (Unified Modelling Language) relevante (precum diagrame de secvență și de activitate) pentru a vizualiza fluxurile și structurile sistemului. Sunt, de asemenea, enumerate tehnologiile și stack-ul de dezvoltare selectate și se descriu principiile de design pentru interfața utilizatorului (UI/UX).

Aspectele practice ale dezvoltării sunt abordate în capitolul de implementare, care descrie structura codului, organizarea în module și componente software, și detaliază fluxul de date principal prin sistem, de la achiziție la prezentare.

Procesul de asigurare a calității este specificat în capitolul de testare, unde se definește strategia de testare, se enumeră tipurile de teste efectuate (unitare, de integrare etc.) și se prezintă un plan de testare cu scenarii specifice.

Considerațiile post-implementare sunt cuprinse în capitolul de deployment și mentenanță, care descrie strategia de deployment a aplicației, instrumentele și metodele de administrare și monitorizare, și un plan de mentenanță pentru actualizări și corecții viitoare.

În final, documentul oferă concluzii asupra procesului de analiză și proiectare, sumarizând principalele constatări și formulând recomandări pentru dezvoltări și îmbunătățiri ulterioare ale aplicației "Market Sentiment Dashboard & AI Analyzer".

2. Analiza cerințelor

2.1 Cerințe funcționale

Enumerarea și descrierea cerințelor funcționale ale aplicației.

Aplicația "Market Sentiment Dashboard & AI Analyzer" trebuie să îndeplinească următoarele cerințe funcționale pentru a asigura o utilizare eficientă și o experiență optimă pentru utilizatori, permițându-le să obțină o perspectivă clară asupra sentimentului pieței financiare:

1. Colectarea Datelor de Piață:
    - 1.1. Agregarea Știrilor din Surse Multiple: Sistemul trebuie să colecteze automat articole de știri financiare relevante pentru piața de acțiuni din SUA din următoarele tipuri de surse.
    - 1.1.1 Fluxuri RSS predefinite (ex: Yahoo Finance News, Investing.com News, Google News pentru site-uri specifice precum Reuters).
    - 1.1.2 API-ul NewsAPI, permițând interogări bazate pe cuvinte cheie specifice domeniului financiar și filtrarea pe surse de încredere (ex: Bloomberg, Reuters, Wall Street Journal).
    - 1.2. Preluarea Indicelui VIX: Sistemul trebuie să preia cea mai recentă valoare disponibilă a indicelui de volatilitate VIX folosind biblioteca yfinance.
    - 1.3 Preluarea Indicelui VIX: Sistemul trebuie să preia cea mai recentă valoare disponibilă a indicelui de volatilitate VIX folosind biblioteca yfinance.

2. Analiza de Sentiment și Procesarea Datelor:
    - 2.1. Preluarea Indicelui VIX: Sistemul trebuie să preia cea mai recentă valoare disponibilă a indicelui de volatilitate VIX folosind biblioteca yfinance.

- 2.2. Generarea Sumarului AI: Pe baza analizei, modelul AI trebuie să genereze un sumar concis al sentimentului general al pieței, evidențiind temele recurente și evenimentele cheie.

- 2.3. Calcularea Indicelui Fear & Greed (F&G): Modelul AI trebuie să estimeze și să returneze o valoare numerică (între 0 și 100) pentru un indice "Fear & Greed" personalizat, bazat pe conținutul știrilor.

- 2.4. Parsarea Răspunsului AI: Sistemul trebuie să parseze răspunsul text de la modelul AI pentru a extrage în mod fiabil sumarul și valoarea numerică a indicelui F&G.

3. Stocarea Datelor și Managementul Istoricului

- 3.1. Persistența Datelor de Sentiment: Rezultatele analizei (valoarea F&G, valoarea VIX, sumarul AI) împreună cu un timestamp UTC trebuie stocate într-o bază de date PostgreSQL (tabela sentiment_history) pentru a crea un istoric.
- 3.2. Stocarea Indicilor Recenți: Cele mai recente valori F&G și VIX, împreună cu timestamp-ul lor, trebuie salvate într-un fișier JSON (latest_indices.json) pentru acces rapid de către interfața web.
- 3.3. Gestionarea Subscrierilor la Alerte VIX: Sistemul trebuie să permită utilizatorilor să se aboneze pentru a primi alerte prin email. Aceste subscrieri (email, prag VIX) trebuie stocate și gestionate în baza de date (tabela vix_alerts_subscriptions).

4. Prezentarea Datelor și Interfața Utilizator (Dashboard)

- 4.1. Afișarea Indicelui F&G Curent: Valoarea curentă a indicelui F&G trebuie afișată vizual printr-un indicator de tip "gauge".
- 4.2. Afișarea Valorii VIX Curente: Valoarea curentă a indicelui VIX trebuie afișată numeric.

- 4.3. Afișarea Sumarului AI: Cel mai recent sumar generat de AI trebuie afișat pe dashboard.
- 4.4. Vizualizarea Trendurilor Istorice: Istoricul valorilor F&G și VIX trebuie prezentat sub formă de grafice de tip linie.
- 4.5. Afișarea Istoricului Recent în Tabel: O listă cu cele mai recente înregistrări de sentiment (timestamp, F&G, VIX) trebuie afișată într-un format tabelar.
- 4.6. Filtrarea Datelor Istorice: Utilizatorii trebuie să poată filtra datele afișate în grafice și tabel pe baza unui interval de date selectat (dată de început, dată de sfârșit).
- 4.7. Afișarea Datei Ultimei Actualizări: Dashboard-ul trebuie să indice clar data și ora ultimei analize de sentiment efectuate.

5. Controlul Utilizatorului și Operațiuni:
    - 5.1. Declanșarea Manuală a Scripturilor: Interfața web trebuie să ofere butoane care permit utilizatorului să declanșeze manual rularea scripturilor de colectare a datelor (webScrape.py) și de analiză (analyze_news.py), precum și a întregului pipeline.
    - 5.2. Exportul Datelor:
    - 5.2.1. Utilizatorii trebuie să poată exporta o imagine a dashboard-ului curent în format PDF.
    - 5.2.2. Utilizatorii trebuie să poată exporta datele istorice (respectând filtrele de dată aplicate) în format CSV.
    - 5.3. Subscrierea la Alerte VIX: Utilizatorii trebuie să poată introduce adresa lor de email și un prag VIX dorit pentru a se abona la notificări prin email. Sistemul trebuie să permită actualizarea pragului pentru o subscriere existentă.

6. Notificări și Alerte (Sistem Automat):
    - 6.1. Monitorizarea VIX pentru Alerte: Un script dedicat (alert_monitor.py) trebuie să ruleze periodic pentru a verifica valoarea curentă a indicelui VIX.Importul și exportul de date între platforme compatibile.
    - 6.2. Trimiterea Alertelor VIX prin Email: Dacă valoarea VIX curentă depășește pragul specificat de un utilizator în subscrierea sa activă (și nu s-a trimis o alertă recent acelui utilizator pentru același eveniment), sistemul trebuie să trimită automat un email de notificare la adresa specificată.

7. Automatizarea Proceselor de Backend:
    - 7.1. Programarea Rulării Scripturilor: Scripturile de colectare a datelor (webScrape.py), analiză (analyze_news.py) și monitorizare a alertelor (alert_monitor.py) trebuie să poată fi rulate automat la intervale de timp configurabile, folosind un scheduler Python (scheduler_main.py).

Aceste cerințe funcționale definesc capabilitățile esențiale ale aplicației și vor fi detaliate ulterior prin cazuri de utilizare și diagrame UML.

2.2 Cerințe non-funcționale

Performanță, scalabilitate, securitate, interoperabilitate etc.

Aplicația "Market Sentiment Dashboard & AI Analyzer" trebuie să îndeplinească următoarele cerințe non-funcționale pentru a asigura o experiență de utilizare calitativă, securitate adecvată și o bună mentenabilitate:

1. **Performanță (optional)**
    - 1.1. Timp de Răspuns al Dashboard-ului: Interfața web principală (dashboard-ul) trebuie să se încarce complet în browserul utilizatorului în mai puțin de 3-4 secunde în condiții normale de rețea și încărcare a serverului.
    - 1.2. Timp de Execuție al Scripturilor de Backend: Scriptul de colectare a datelor (webScrape.py) ar trebui să își finalizeze execuția într-un interval de timp rezonabil (ex: sub 5-10 minute), depinzând de numărul de surse RSS și de răspunsul API-urilor externe.
    - 1.2.1. Scriptul de analiză AI (analyze_news.py) ar trebui să proceseze datele și să interacționeze cu API-ul Azure într-un timp optimizat, fără a cauza întârzieri excesive în actualizarea datelor. Timpul de răspuns al API-ului Azure este o dependență externă.
    - 1.2.2. Scriptul de monitorizare a alertelor VIX (alert_monitor.py) trebuie să ruleze rapid pentru a verifica subscrierile și a trimite notificările (ex: sub 1 minut per ciclu de verificare, excluzând timpul de preluare VIX).
    - 1.3. Eficiența Interogărilor DB: Interogările către baza de date PostgreSQL pentru afișarea datelor pe dashboard și pentru exporturi trebuie optimizate pentru a returna rezultate rapid (ex: prin utilizarea de indecși pe coloanele frecvent filtrate sau sortate, cum ar fi timestamp).

2. **Scalabilitate (optional)**

   - Scalabilitatea Bazei de Date: Arhitectura bazei de date PostgreSQL permite gestionarea unui volum considerabil de înregistrări istorice (ex: sute de mii) fără degradare semnificativă a performanței interogărilor, prin utilizarea corectă a indecșilor.Arhitectura trebuie să permită extinderea resurselor (scalare orizontală și verticală) fără întreruperi majore.
   - Scalabilitatea Componentelor: Designul modular al scripturilor Python (colectare, analiză, alerte, web) permite, teoretic, scalarea independentă a acestor componente dacă aplicația ar fi deployată într-un mediu cloud cu capabilități de scalare orizontală (ex: mai mulți workeri pentru scripturile de backend).

3. **Securitate**

   - Protecția Credentialelor: Toate cheile API (NewsAPI, Azure AI) și credentialele de acces la baza de date și serverul SMTP trebuie stocate în mod securizat ca variabile de mediu și nu trebuie hardcodate în codul sursă. Conexiunea la API-ul Azure AI se realizează prin HTTPS. Conexiunea la serverul SMTP(serviciul de mail) pentru trimiterea emailurilor de alertă trebuie să utilizeze criptare (TLS/STARTTLS).
   - Validarea Input-ului (Formular Subscriere): Datele introduse de utilizator în formularul de subscriere la alerte VIX (email, prag) trebuie validate pe server pentru a preveni date incorecte sau potențiale atacuri simple (ex: lungime, format email). (Asigurat prin Flask-WTF).
   - Prevenirea SQL Injection: Interogările SQL către baza de date trebuie să utilizeze interogări parametrizate pentru a preveni vulnerabilitățile de tip SQL Injection. (Asigurat prin psycopg2 când se folosesc placeholder-e %s).

4. **Fiabilitate și Disponibilitate:**

- Gestionarea Erorilor în Scripturi: Scripturile de backend (webScrape.py, analyze_news.py, alert_monitor.py) trebuie să includă mecanisme de gestionare a erorilor (ex: try-except) pentru a trata probleme tranzitorii (ex: erori de rețea la apelarea API-urilor, probleme temporare de conexiune la DB) și pentru a loga aceste erori, permițând continuarea funcționării sau oprirea grațioasă.Compatibilitate cu baze de date SQL și NoSQL pentru gestionarea eficientă a datelor.
- Mecanisme de Retry: Scriptul webScrape.py include un mecanism de reîncercare pentru preluarea datelor de la URL-uri externe.
- Disponibilitatea Dashboard-ului: Aplicația web Flask, atunci când este deployată corespunzător (ex: cu un server WSGI de producție), ar trebui să aibă o disponibilitate ridicată. (Pentru contextul proiectului de laborator, rularea pe serverul de dezvoltare Flask este suficientă).

5. **Mentenabilitate:**

- Lizibilitatea Codului: Codul sursă trebuie să fie clar, bine structurat și comentat corespunzător pentru a facilita înțelegerea și modificările ulterioare.
- Modularitate: Aplicația este împărțită în module distincte (colectare date, analiză AI, alerte, interfață web, scheduler) cu responsabilități clare, facilitând dezvoltarea și depanarea independentă a componentelor.
- Configurabilitate: Parametrii cheie ai aplicației (chei API, detalii DB, praguri, surse de știri) sunt configurabili prin variabile de mediu sau constante definite la începutul scripturilor.

6. **Ușurință în utilizare(UI/UX Dashboard)**

- Intuitivitate: Interfața web trebuie să fie intuitivă, permițând utilizatorilor să înțeleagă rapid informațiile prezentate și să acceseze funcționalitățile disponibile (filtrare, export, rulare scripturi, subscriere alerte).Sistemul trebuie să includă documentație și suport pentru utilizatori (opțional).
- Consistență Vizuală: Designul trebuie să fie consistent pe întreaga pagină, folosind o paletă de culori și o tipografie unitară.
- Feedback Utilizator: Aplicația trebuie să ofere feedback utilizatorului pentru acțiunile efectuate (ex: mesaje de succes/eroare la subscrierea la alerte, indicarea stării de încărcare la rularea scripturilor din UI).
- Responsivitate: Dashboard-ul trebuie să fie responsiv și utilizabil pe diferite dimensiuni de ecran (desktop, tabletă, mobil).

Prin îndeplinirea acestor cerințe non-funcționale, se urmărește asigurarea unei aplicații performante, sigure și ușor de utilizat, care să ofere o experiență valoroasă și fiabilă utilizatorilor interesați de dinamica sentimentului pieței financiare.

2.3 Cazuri de utilizare (Use Cases)

Descrierea scenariilor de utilizare cu diagrame UML corespunzătoare.

Această secțiune prezintă principalele cazuri de utilizare ale aplicației "Market Sentiment Dashboard & AI Analyzer", ilustrând interacțiunile cheie ale utilizatorilor și sistemului. Diagramele UML corespunzătoare (diagrame de cazuri de utilizare, diagrame de secvență) vor fi prezentate în capitolul de Arhitectură și Proiectare.

UC1: Vizualizare Dashboard Sentiment Piață

Descriere: Un utilizator accesează aplicația web pentru a vizualiza starea curentă și istoricul sentimentului pieței financiare.

Actori: Utilizator (ex: Investitor, Analist Amator).

Precondiții: Aplicația Flask rulează. Scripturile de colectare și analiză a datelor au rulat cel puțin o dată și există date în baza de date.

Pași:

1. Utilizatorul navighează la URL-ul principal al aplicației în browserul web.Introduce numele de utilizator și parola.
2. Sistemul (Aplicația Flask) preia cele mai recente date de sentiment (F&G Index, VIX, sumar AI, data ultimei actualizări) din baza de date sau fișierul cache (latest_indices.json).Dacă autentificarea este reușită, utilizatorul este direcționat către pagina principală.
3. Sistemul preia datele istorice pentru F&G Index și VIX din baza de date, conform limitelor default pentru grafice și tabel.
4. Sistemul randează pagina principală (dashboard-ul) și o trimite browserului.l randează pagina principală
5. Browserul afișează dashboard-ul.

UC2: Filtrare Date Istorice pe Dashboard

Descriere: Utilizatorul aplică filtre de dată pentru a vizualiza un anumit interval din istoricul de sentiment.

Actori: Utilizator

Precondiții: Utilizatorul se află pe pagina principală a dashboard-ului (UC1).

Pași:

1. Utilizatorul selectează o dată de început și/sau o dată de sfârșit în formularul de filtrare.Selectează opțiunea de creare, editare sau ștergere utilizator.
2. Utilizatorul apasă butonul "Filter".
3. Browserul trimite un request GET către server cu noii parametri de dată în URL
4. Sistemul (Aplicația Flask) preia datele istorice din baza de date, filtrate conform noului interval de date.
5. Sistemul re-randează pagina principală cu graficele și tabelul actualizate pentru a reflecta intervalul selectat.
6. Câmpurile de dată din formular sunt pre-populate cu valorile selectate

Post-condiții: Dashboard-ul afișează datele istorice pentru intervalul de timp specificat.

UC3: Export Date Istorice în format CSV

Descriere: Utilizatorul descarcă istoricul datelor de sentiment (filtrat sau complet) într-un fișier CSV.

Actori: Utilizator, Sistem de Raportare

Precondiții: Utilizatorul se află pe pagina principală a dashboard-ului.

Pași:

1. Utilizatorul apasă butonul/link-ul "Export CSV".
2. Browserul trimite un request GET către o rută Flask dedicată (/export/csv), incluzând eventualii parametri de filtrare pe dată activi.Sistemul procesează cererea și generează raportul.
3. Sistemul (Aplicația Flask) preia datele istorice relevante din baza de date, conform filtrelor.
4. Sistemul generează conținutul fișierului CSV în memorie.
5. Sistemul returnează un răspuns HTTP cu mimetype='text/csv' și header-ul Content-Disposition setat pentru descărcare, declanșând descărcarea fișierului sentiment_history.csv în browserul utilizatorului.

Post-condiții: Utilizatorul a descărcat un fișier CSV cu datele istorice.

UC4: Export Dashboard în format PDF

Descriere: Utilizatorul descarcă o imagine a stării curente a dashboard-ului într-un fișier PDF.

Actori: Utilizator.

Precondiții: Utilizatorul se află pe pagina principală a dashboard-ului.

Pași (Flux Principal):

1. Utilizatorul apasă butonul "Export PDF".
2. JavaScript-ul din browser (folosind html2canvas) capturează o imagine a conținutului relevant al dashboard-ului.
3. JavaScript-ul (folosind jsPDF) inserează imaginea capturată într-un nou document PDF.
4. JavaScript-ul declanșează descărcarea fișierului PDF generat în browserul utilizatorului.

Postcondiții: Utilizatorul a descărcat un fișier PDF cu o imagine a dashboard-ului.

Fluxuri Alternative/Excepții:

4a. Eroare la generarea imaginii sau PDF-ului: JavaScript afișează un mesaj de eroare utilizatorului.

UC5: Subscripție la Alerte VIX

Descriere: Un utilizator își introduce adresa de email și un prag VIX pentru a primi notificări când VIX-ul depășește acel prag.

Actori: Utilizator.

Precondiții: Utilizatorul se află pe pagina principală a dashboard-ului (unde se află formularul de subscriere).

Pași (Flux Principal):

1. Utilizatorul completează câmpurile "Email Address" și "Alert me when VIX is above" din formularul de subscriere.
2. Utilizatorul apasă butonul "Subscribe / Update Alert".
3. Browserul trimite datele formularului printr-un request POST către server (Aplicația Flask).
4. Sistemul validează datele primite (format email, valoare numerică pozitivă pentru prag).
5. Dacă datele sunt valide:
   a. Sistemul verifică dacă emailul există deja în tabela vix_alerts_subscriptions.
   b. Dacă există, actualizează pragul VIX și reactivează subscrierea (setează is_active = TRUE).
   c. Dacă nu există, inserează o nouă subscriere în tabelă.

d. Sistemul afișează un mesaj de succes utilizatorului (folosind flash).

6. Sistemul face redirect înapoi la pagina principală.

Postcondiții: Subscrierea utilizatorului este înregistrată/actualizată în baza de date. Utilizatorul este notificat despre succesul operațiunii.

Fluxuri Alternative/Excepții:

5a. Date invalide în formular: Sistemul afișează mesaje de eroare specifice lângă câmpurile invalide și nu procesează subscrierea.

5b. Eroare la comunicarea cu baza de date: Sistemul afișează un mesaj de eroare general utilizatorului.

UC6: Rulare Manuală Scripturi Backend din UI

Descriere: Un utilizator (probabil dezvoltatorul/administratorul) declanșează rularea scripturilor de colectare sau analiză direct din interfața web.

Actori: Utilizator (cu rol implicit de admin/developer în acest context).

Precondiții: Aplicația Flask rulează.

Pași (Flux Principal pentru "Refresh All Data"):

1. Utilizatorul apasă butonul "Refresh All Data".
2. Browserul cere confirmarea utilizatorului.
3. Dacă se confirmă, JavaScript trimite un request POST către ruta Flask /run_pipeline.
4. Sistemul (Aplicația Flask):
    a) Execută webScrape.py ca un subproces. Așteaptă finalizarea.
    b) Dacă webScrape.py rulează cu succes, execută analyze_news.py ca un subproces. Așteaptă finalizarea.
    c) (Implicit) Scriptul alert_monitor.py va rula conform programării sale separate (sau ar putea fi adăugat și aici).
5. Sistemul returnează un mesaj JSON către browser indicând succesul sau eșecul operațiunii.
6. JavaScript afișează un alert() cu mesajul de la server și, în caz de succes, reîncarcă pagina după un scurt delay.

Postcondiții: Scripturile specificate au fost rulate pe server. Datele de pe dashboard sunt (eventual) actualizate la următorul refresh al paginii.

Fluxuri Alternative/Excepții:

6a. Un script eșuează: Sistemul returnează un mesaj de eroare specific. Pipeline-ul se poate opri (ex: analyze_news nu rulează dacă webScrape eșuează).

6b. Utilizatorul anulează confirmarea: Acțiunea nu este executată.

(Similar pentru butoanele "Run Scraper" și "Run Analyzer" care apelează rutele lor specifice).

UC7: Trimitere Automată Alerte VIX (Proces de Backend)

Descriere: Sistemul monitorizează periodic valoarea VIX și trimite automat emailuri de alertă utilizatorilor abonați al căror prag a fost depășit.

Actori: Sistem (Scriptul alert_monitor.py rulat de scheduler_main.py).

Precondiții: Scriptul scheduler_main.py rulează și este programat să execute alert_monitor.py. Există subscrieri active în tabela vix_alerts_subscriptions. Variabilele de mediu SMTP sunt configurate.

Pași (Flux Principal):

1. Scheduler-ul (scheduler_main.py) declanșează rularea scriptului alert_monitor.py.
2. alert_monitor.py preia valoarea curentă a indicelui VIX (folosind get_vix_value_yfinance).

3. Dacă VIX-ul este obținut cu succes:

a. Scriptul interoghează tabela vix_alerts_subscriptions pentru a găsi toate subscrierile active unde VIX-ul curent este mai mare decât vix_threshold al utilizatorului ȘI ultima alertă a fost trimisă cu mai mult de X ore în urmă (sau nu a fost trimisă niciodată).

b. Pentru fiecare subscriere eligibilă, scriptul construiește și trimite un email de notificare la adresa specificată.

c. Dacă emailul este trimis cu succes, scriptul actualizează câmpul last_alert_sent_at pentru acea subscriere în baza de date.

Postcondiții: Utilizatorii eligibili au primit emailuri de alertă VIX. Timestamp-ul ultimei alerte este actualizat în baza de date.

- **Fluxuri Alternative/Excepții:**
    - **7a. Nu se poate prelua valoarea VIX:** Scriptul loghează eroarea și se oprește pentru ciclul curent.
    - **7b. Eroare la conectarea/interogarea DB:** Scriptul loghează eroarea.
    - **7c. Eroare la trimiterea unui email:** Scriptul loghează eroarea pentru acel email specific și continuă cu restul subscrierilor (dacă este posibil). last_alert_sent_at nu este actualizat pentru subscrierea eșuată.
    - **7d. Nicio subscriere eligibilă:** Scriptul se termină fără a trimite emailuri.

2.4 Constrângeri

Limitări tehnice, legale sau operaționale.

Această secțiune identifică principalele constrângeri tehnice, legale și operaționale care pot afecta dezvoltarea și implementarea aplicației.

1. **Constrângeri tehnice**

   - CT1 Dependența de Servicii Externe și API-uri: Funcționalitatea de bază a aplicației depinde în mod critic de disponibilitatea și termenii de utilizare ai serviciilor externe:

     a) NewsAPI: Pentru agregarea unei părți semnificative a știrilor. Planul gratuit are limitări privind numărul de request-uri și istoricul datelor.
     b) Azure AI Services (pentru modelul DeepSeek): Pentru analiza de sentiment și generarea indicelui F&G. Disponibilitatea și costurile asociate cu acest serviciu sunt factori externi. Orice modificare a API-ului sau a modelului poate necesita actualizări ale codului.
     c) Servicii RSS Feeds: Disponibilitatea și formatul fluxurilor RSS pot varia și nu sunt sub controlul proiectului.
     d) yfinance Library: Datele de la Yahoo Finance (în special pentru VIX) pot avea întârzieri și fiabilitatea bibliotecii poate depinde de schimbările neanunțate ale API-ului Yahoo Finance.

   - CT2 Resurse de Calcul pentru Modelul AI: Deși modelul DeepSeek este accesat ca serviciu prin Azure, generarea răspunsurilor (mai ales cele lungi solicitate) poate consuma resurse și poate avea o latență care influențează timpul total de procesare al scriptului analyze_news.py.Baza de date trebuie să suporte minim 100.000 de înregistrări fără degradarea performanței.

   - CT3 Mediu de Rulare Python: Aplicația și scripturile asociate sunt dezvoltate în Python și necesită un mediu Python configurat corespunzător cu toate bibliotecile.

   - CT4 Baza de Date PostgreSQL: Aplicația necesită o instanță PostgreSQL funcțională și accesibilă pentru stocarea datelor istorice și a subscrierilor la alerte.

   - CT5 Conexiune la Internet: Toate funcționalitățile de colectare a datelor, analiză AI și trimitere a alertelor email necesită o conexiune la internet activă pentru serverul/mașina care rulează scripturile.

   - CT6 Limitări ale Browserului Client (pentru Export PDF): Funcționalitatea de export PDF client-side (cu html2canvas și jsPDF) depinde de capabilitățile browserului utilizatorului și poate avea limitări de performanță sau calitate pentru dashboard-uri foarte complexe.

2. **Constrângeri legale și de Conformitate**

   - CL1 Termenii de Utilizare ai API-urilor: Aplicația trebuie să respecte termenii și condițiile de utilizare impuse de furnizorii de API-uri (NewsAPI, Azure AI Services, Yahoo Finance prin yfinance). Aceasta include respectarea limitelor de request-uri, atribuirea corectă (dacă este cerută) și utilizarea non-comercială (dacă se aplică planurilor gratuite).Respectarea standardelor ISO/IEC 27001 privind securitatea informațiilor (opțional)

   - CL2 Drepturi de Autor pentru Conținutul Știrilor: Aplicația agregă și procesează conținut (titluri, sumare scurte) din diverse surse. Deși nu stochează conținutul complet al articolelor pe termen lung, trebuie avută în vedere respectarea drepturilor de autor. Sumarele generate de AI sunt derivate, dar se bazează pe conținut original.
   - CL3 Protecția Datelor Personale (GDPR - pentru Alerte Email): Prin colectarea adreselor de email pentru subscrierile la alerte VIX, aplicația gestionează date cu caracter personal. Trebuie asigurată o informare minimă a utilizatorului despre scopul colectării și (ideal) o modalitate de dezabonare/ștergere a datelor. (Pentru un proiect de laborator, implementarea completă GDPR poate fi în afara scopului, dar conștientizarea este importantă).

3. **Constrângeri operaționale**

   - CO1 Necesitatea Rulării Periodice a Scripturilor: Pentru ca datele de pe dashboard și alertele VIX să fie actuale, scripturile (webScrape.py, analyze_news.py, alert_monitor.py) trebuie rulate periodic. Acest lucru este gestionat prin scheduler_main.py, care necesită ca mașina pe care rulează să fie pornită și scriptul scheduler să fie activ.
   - CO2 Managementul Cheilor API și Credentialelor: Securitatea cheilor API și a credentialelor de acces la baza de date și SMTP este crucială. Acestea trebuie gestionate prin variabile de mediu și nu expuse în codul sursă.
   - CO3 Trimiterea Emailurilor și Filtrele de Spam: Livrarea emailurilor de alertă poate fi afectată de filtrele de spam ale providerilor de email ai destinatarilor. Conținutul și formatarea emailului, precum și reputația serverului SMTP (în acest caz, Gmail) pot influența acest aspect.
   - CO4 Resurse Limitate pentru Dezvoltare și Testare: Fiind un proiect de laborator, timpul și resursele pentru testare exhaustivă pe multiple platforme sau pentru optimizări avansate de performanță sunt limitate.

2.5 Riscuri

Identificarea riscurilor și modalitățile de gestionare a acestora.

Această secțiune identifică principalele constrângeri tehnice, legale și operaționale care pot afecta sau au influențat dezvoltarea și implementarea aplicației "Market Sentiment Dashboard & AI Analyzer".secțiune identifică principalele riscuri asociate cu dezvoltarea și implementarea aplicației, precum și modalitățile de gestionare a acestora.

1. **Riscuri tehnice:**

- RT1: Indisponibilitatea sau Modificarea API-urilor Externe:
  a) Descriere: Aplicația depinde de NewsAPI, Azure AI Services (DeepSeek), fluxuri RSS și yfinance. Aceste servicii pot deveni temporar indisponibile, își pot schimba termenii de utilizare, structura API-ului, sau pot introduce/modifica limitări de acces.
  b) Măsură de atenuare: Implementarea mecanismelor de retry cu backoff exponențial pentru apelurile API (parțial implementat în webScrape.py). Logarea detaliată a erorilor de la API-uri pentru diagnosticare rapidă. Monitorizarea periodică a statusului API-urilor și adaptarea codului la schimbări. Prevederea unor valori default sau mesaje de eroare clare în UI când datele nu pot fi preluate.

- RT2: Calitatea și Relevanța Datelor Agregate:
  a) Descriere: Calitatea analizei AI depinde de calitatea știrilor agregate. Sursele RSS pot conține zgomot sau articole irelevante. Query-urile NewsAPI pot returna articole mai puțin pertinente.
  b) Măsură de atenuare: Rafinarea constantă a query-urilor NewsAPI și a listei de surse RSS. Implementarea unor filtre suplimentare în webScrape.py pentru a exclude articolele care nu par relevante (bazat pe cuvinte cheie, lungime etc.).

- RT3: Performanța și Costurile Serviciului AI:
  a) Descriere: Apelurile către modelul DeepSeek pe Azure pot avea o latență variabilă și pot genera costuri dacă se depășește un eventual

tier gratuit sau limitele alocate. Solicitarea unor răspunsuri foarte lungi crește latența și consumul de tokeni.

b) Măsură de atenuare: Optimizarea prompt-urilor trimise la AI. Setarea unui MAX_TOKENS realist pentru răspunsul AI, conform limitelor modelului și nevoilor aplicației. Monitorizarea utilizării și costurilor Azure (dacă este cazul). Considerarea unor mecanisme de caching pentru analize pe seturi de știri similare (complex, pentru viitor).

- RT4: Erori de Parsare a Răspunsului AI:
  a) Descriere: Structura răspunsului text de la AI (în special linia F&G Index) se poate modifica ușor, ducând la eșecul parsării cu expresii regulate.
  b) Măsură de atenuare: Crearea unor expresii regulate cât mai robuste și flexibile. Implementarea unei logici de fallback (ex: setarea F&G la None și logarea erorii) dacă parsarea eșuează, pentru a nu bloca întregul flux. Testare riguroasă a funcției de parsare cu diverse exemple de output AI.

- RT5: Probleme de Encoding al Datelor:
  a) Descriere: Caractere speciale din știri sau din răspunsul AI pot cauza UnicodeDecodeError dacă encoding-urile nu sunt gestionate corect la citire/scriere fișiere sau la capturarea output-ului subproceselor.
  b) Măsură de atenuare: Utilizarea consistentă a encoding='utf-8' la deschiderea și scrierea fișierelor. Folosirea errors='replace' la decodarea output-ului subproceselor în scheduler_main.py și appFlask.py

2. **Riscuri de Securitate:**

- **RS1: Expunerea Cheilor API și Credentialelor**

a) Descriere: Stocarea necorespunzătoare a cheilor API (NewsAPI, Azure) sau a credentialelor DB/SMTP în codul sursă le poate expune.
b) Măsură de atenuare: Stocarea tuturor cheilor și credentialelor în variabile de mediu ale sistemului de operare, neincluse în repository-ul Git (prin .gitignore dacă s-ar folosi un fișier .env).

- RS2: Vulnerabilități în Formularul de Subscriere Alerte:
  a) Descriere: Formularul web pentru subscrierea la alerte VIX ar putea fi vulnerabil la atacuri simple (ex: Cross-Site Scripting - XSS) dacă input-ul nu este validat și sanitizat corespunzător.
  b) Măsură de atenuare: Utilizarea Flask-WTF pentru validarea datelor din formular pe server (format email, tip numeric pentru prag) și pentru protecția CSRF. Escaparea output-ului în template-urile Jinja2 (este comportamentul default).

- RS3: Acces Neautorizat la Funcționalitățile de Rulare a Scripturilor (din UI):
  a) Descriere: Butoanele care declanșează scripturi pe server ar putea fi abuzate dacă nu există un mecanism de control al accesului.
  b) Măsură de atenuare: Pentru un proiect de laborator, acest risc este acceptat. Într-o aplicație de producție, aceste rute ar trebui protejate prin autentificare și autorizare (ex: doar pentru administratori).

3. **Riscuri de proiect**

- RP1: Depășirea Termenelor Limită:

  a) Descriere: Complexitatea integrării multiplelor servicii și funcționalități (colectare date, AI, DB, web, alerte, scheduler, teste) poate duce la întârzieri.

  b) Măsură de atenuare: Planificare iterativă, prioritizarea funcționalităților cheie. Împărțirea task-urilor în unități mai mici și gestionabile. Comunicare constantă (dacă ar fi un proiect de echipă).

- RP2: Dependența de Resurse Limitate (Conturi Gratuite API):

  a) Descriere: Utilizarea planurilor gratuite pentru NewsAPI sau Azure AI poate impune limite stricte care afectează testarea extensivă sau rularea frecventă a scripturilor.

  b) Măsură de atenuare: Optimizarea numărului de apeluri API. Programarea rulărilor automate la intervale care respectă aceste limite. Testare focusată pe funcționalitățile cheie.

4. **Riscuri operaționale**

- RO1: Eșecul Scripturilor Automate (Scheduler):

  a) Descriere: Scriptul scheduler_main.py sau scripturile individuale pe care le rulează pot eșua din diverse motive (erori de cod neanticipate, probleme de rețea persistente, modificări în API-uri externe negestionate).

  b) Măsură de atenuare: Logare detaliată a output-ului și erorilor pentru fiecare script rulat de scheduler (implementat prin scrierea în fișiere de log). Monitorizarea periodică a acestor log-uri. Implementarea unor mecanisme de notificare (email) pentru administrator în caz de eșecuri critice repetate (ca "Future Improvement").

- RO2: Livrarea Emailurilor de Alertă în Spam:

  a) Descriere: Emailurile de alertă VIX pot ajunge în folderul Spam al destinatarilor.

  b) Descriere: Emailurile de alertă VIX pot ajunge în folderul Spam al destinatarilor.

- RO3: Fiabilitatea Datelor VIX de la yfinance:
    a) Descriere: Biblioteca yfinance se bazează pe scraping neoficial al Yahoo Finance, care se poate schimba fără preaviz, afectând preluarea datelor VIX.
    b) Măsură de atenuare: Includerea de blocuri try-except robuste în jurul apelurilor yfinance. Logarea erorilor. Considerarea unor surse alternative pentru VIX ca "Future Improvement" dacă fiabilitatea devine o problemă majoră.

3. Arhitectura și proiectarea sistemului

3.1 Arhitectura generală

Prezentarea arhitecturii software (de ex. client-server, microservicii, MVC).

Aplicația este construită pe o arhitectură pe niveluri (Layered Arhitecture), utilizând un model client-server pentru componenta web și un flux de procesare secvențial pentru colectarea și analiza datelor. Această abordare modulară facilitează separarea responsabilităților și mentenanța. Arhitectura este împărțită conceptual în următoarele niveyluri principale.

1. **Nivelul de prezentare (Frontend):**
    - Implementat folosind **Flask (framework Python) pentru a servi conținutul dinamic și a gestiona interacțiunile utilizatorului.**
    - Interfața utilizatorul este construită cu HTML5, CSS3 și JavaScript(ES6+).
    - Pentru vizualizări de date interactive, se utilizează bibliotecile JavaScript Gauge.js(pentru indicatorul F&G) și Chart.js (pentru graficele istorice)
    - Funcționalitatea de export PDF client-side este realizată cu html2canvas și jsPDF.
    - Comunică cu nivelul de aplicație/backend prin request-uri HTTP (GET pentru afișarea datelor, POST pentru acțiuni precum subscrierea la alerte sau declanșarea scripturilor).

2. **Nivelul de logică de business (Backend – Flask și Scripturi Python)**
    - appFlask.py: acționează ca principal controller web. Gestionează rutele HTTP, preia datele de la utilizator(filtre, subscrieri), interacționează cu nivelul de date pentru a prelua informațiile de sentiment, și declanșează execuția scripturilor de backend prin subprocese.
    - webScrape.py: Script Python responsabil penntru colectarea datelor din surse externe (fluxuri RSS folosind feedparser, NewsAPI folosind newsapi-python, și date VIX de pe yahoo finance folosind yfinance). Acest script agregă și pre-procesează aceste date.
    - analyze_news.py: script Python care preia datele agregate de webScrape.py, interacționează cu serviciul Azure AI(pentru LLM DeepSeek R1) prin SDK-ul azure-ai-inference pentru a obține analiza de sentiment și indicele F&G, și apoi salvează rezultatele în baza de date și într-un fișier JSON cache.
    - alert_monitor.py: script Python care preia valoarea VIX, verifică subscripțiile utilizatorilor din baza de date și trimite alerte email prin SMTP (folosind smtplib) dacă pragurile VIX sunt depășite.
    - scheduler_main.py: script Python care utilizează biblioteca schedule pentru a orchestra rularea periodică (la 25 de minute, dar se poate seta și mai frecvent întrucât NewsAPI are o limită de maximum 100 de API calls pe zi) și automată a scripturilor webScrape.py, analyze_news.py și alert_monitor.py.
3. **Nivelul de date:**
    - PostgreSQL: utilizat ca sistem de management al bazelor de date relaționale pentru stocarea datelor structurate. În el există 2 tabele: tabela sentiment_history ce stocheaza istoricul analizelor de sentiment(F&G; VIX, sumar AI și timestamp) și tabela vix_alerts_subscriptions ce stocheaza abonamentele utilizatorilor la alertele VIX(email, prag VIX)
    - Pyscopg2-binary: adaptorul Python folosit pentru a interacționa cu baza de date PostgreSQL
    - Fișiere JSON: financial_news_agg.json: folosit ca stocare intermediară pentru datele agregate de webScrape.py înainte de a fi procesate de analyze_news.py. Fișierul latest_indices.json este folosit pentru a stoca cele mai recente valori F&G și VIX pentru acces rapid de către appFlask.py, e mai mult utilizat pentru verificare întrucât appFlask.py în versiunea curentă preia direct din DB.

4. **Securitate:**
    - Variabilele de Mediu: Cheile API (NewsAPI, Azure AI API) și credidențialele pentru baza de date și serverul SMTP sunt gestionate prin variabile de mediu ale sistemului de operare pentru anu fi expuse în cod.
    - HTTPS/TLS: Se presupune că interacțiunea cu API externe se face prin HTTPS. Trimiterea email-urilor prin SMTP este configurată să folosească STARTTLS pentru criptare.
    - Prevenire SQL Injection: interogările către PostgreSQL sunt realizate folosind interogări parametrizate prin psycopg2.

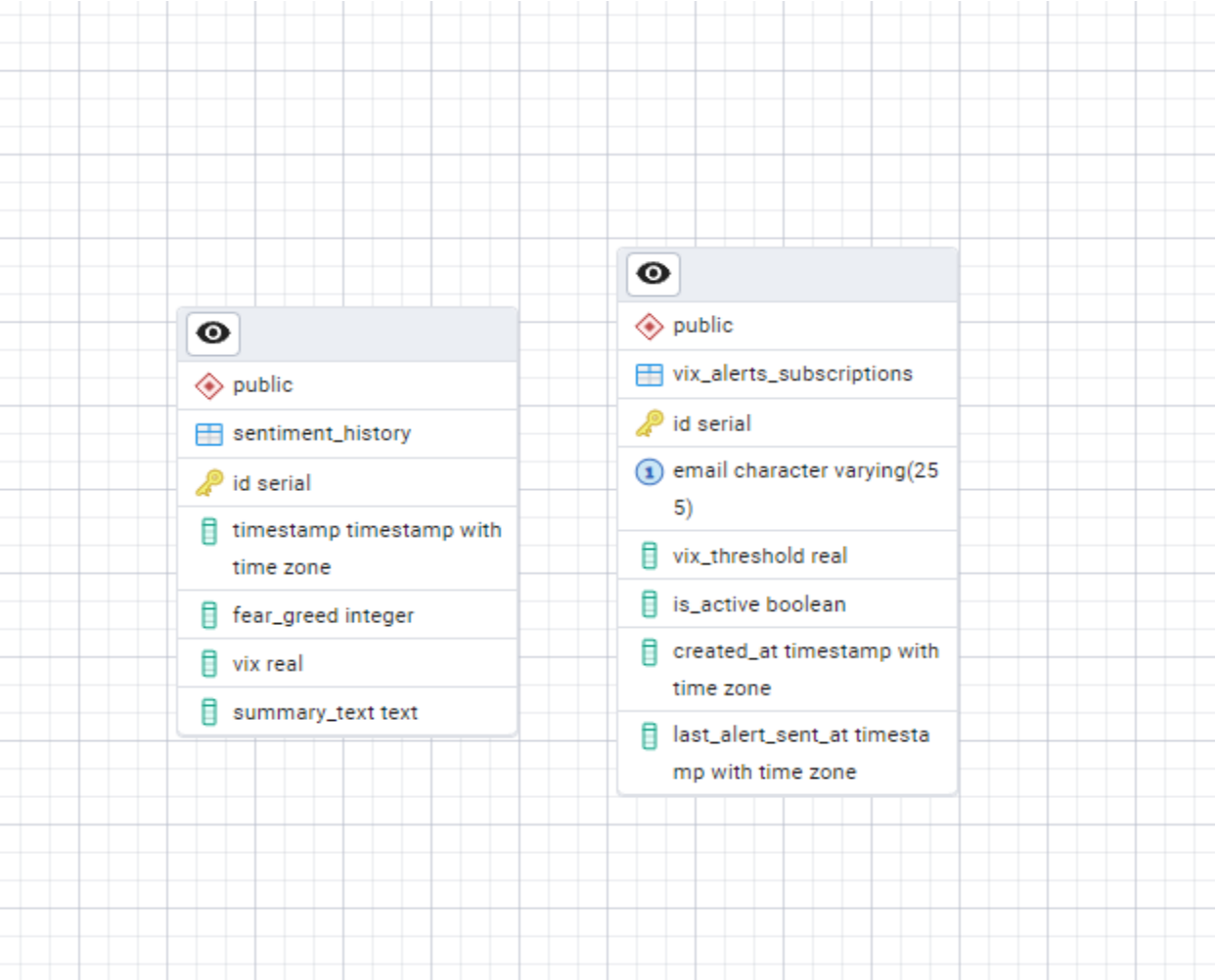
5. **Deployment și scalabilitate:**
    - Aplicația este concepută pentru a rula local. Serverul de dezvoltare Flask este folosit pentru componenta web. Scripturile de backend și scheduler-ul rulează ca procese Python separate.
    - OPȚIONAL: Aplicația ar putea fi containerizată folosindă Docker pentru a simplifica deployment-ul pe alte medii. Totodată, se poate configura un pipeline CI/CD cu ajutorul GitHub Actions pentru automatizarea testării și a procesului de build.
    - Pentru o scalabilitate mai mare într-un mediu de producție, s-ar putea considera deployarea aplicației Flask cu un server WSGI(ex. Gunicorn) în spatele unui reverse proxy(ex. Nginx) și rularea scripturilor de backend ca task-uri gestionate de un sistem de cozi de mesaje(ex. Celery) sau instanțe separate.

3.2 Modelul de date

Diagrama entitate-relație (ERD), descrierea tabelelor și relațiilor.

Această secțiune prezintă modelul de date utilizat în aplicația "Market Sentiment Dashboard & AI Analyzer", incluzând structura bazei de date PostgreSQL și descrierea tabelelor principale. Modelul este conceput pentru a stoca eficient datele istorice de sentiment și preferințele utilizatorilor pentru alerte.

Diagrama Entitate-Relatie (ERD)



Figură 1.1

Diagrama ERD prezintă principalele entități și relațiile dintre ele:

1. Sentiment_history
2. **Vix_alerts_subscriptions**

Descrierea entităților și relațiilor

- **Tabela sentiment_history are ca scop stocarea înregistrărilor istorice ale analizei de sentiment și valorile VIX corespunzătoare. Fiecare rând reprezintă o analiză efectuată la un anumit moment. Are ca atribute:**
    - **id (SERIAL, PRIMARY KEY): Identificator unic auto-incrementat pentru fiecare înregistrare de sentiment.**
    - **timestamp (TIMESTAMPTZ, DEFAULT CURRENT_TIMESTAMP): Data și ora exactă (cu fus orar, implicit UTC) la care a fost efectuată și înregistrată analiza. Acest câmp este indexat pentru interogări rapide bazate pe timp.**
    - **fear_greed (INTEGER, NULLABLE): Valoarea numerică (0-100) a indicelui Fear & Greed estimat de modelul AI. Poate fi NULL dacă analiza F&G a eșuat.**
    - **vix (REAL, NULLABLE): Valoarea numerică a indicelui VIX la momentul respectiv. Poate fi NULL dacă preluarea VIX a eșuat.**
    - **summary_text (TEXT, NULLABLE): Sumarul text al sentimentului pieței generat de modelul AI. Poate fi NULL dacă analiza textului a eșuat.**
- **Tabela vix_alerts_subscriptions are ca scop stocarea informațiilor despre utilizatorii care s-au abonat pentru a primi alerte prin email atunci când indicele VIX depășește pragul setat de utilizator. Are ca atribute:**
    - **id (SERIAL, PRIMARY KEY): Identificator unic auto-incrementat pentru fiecare subscriere.**
    - **email (VARCHAR(255), UNIQUE, NOT NULL): Adresa de email a utilizatorului care s-a abonat. Este unică pentru a preveni subscrieri multiple cu același email și este indexată pentru căutări rapide.**
    - **vix_threshold (REAL, NOT NULL): Valoarea numerică a pragului VIX setat de utilizator. Alerta se va declanșa dacă VIX-ul curent depășește această valoare.**
    - **is_active (BOOLEAN, DEFAULT TRUE, NOT NULL): Un flag care indică dacă subscrierea este activă. Permite implementarea funcționalității de dezabonare fără a șterge înregistrarea.**
    - **created_at (TIMESTAMPTZ, DEFAULT CURRENT_TIMESTAMP): Data și ora la care a fost creată sau actualizată ultima dată subscrierea.**

- **last_alert_sent_at (TIMESTAMPTZ, NULLABLE): Data și ora la care a fost trimisă ultima alertă pentru această subscriere. Folosit pentru a preveni trimiterea prea frecventă a alertelor.**

În modelul actual, tabelele sentiment_history și vix_alerts_subscriptions sunt independente și nu au chei străine (foreign keys) care să le lege direct.

Logica de corelare între ele se face la nivel de aplicație (în scriptul alert_monitor.py), unde valoarea VIX curentă (care ar putea fi similară cu ultima valoare din sentiment_history sau preluată în timp real) este comparată cu vix_threshold din vix_alerts_subscriptions.

3.3 Diagrame UML (Unified Modelling Language)

Această secțiune prezintă principalele diagrame UML utilizate pentru a modela vizual diverse aspecte ale aplicației "Market Sentiment Dashboard & AI Analyzer", incluzând structura, interacțiunile și fluxurile de proces.

1. Diagrama de Clase (Simplificată)

Diagrama de clase pentru această aplicație se concentrează pe modulele software principale și pe entitățile de date cheie, mai degrabă decât pe o ierarhie complexă de clase orientate pe obiect.

- **Module Principale (pot fi reprezentate ca și clase/componente într-o diagramă de componente sau pachete):**
  - **WebScrapeModule: Reprezintă logica din webScrape.py (colectare știri, VIX).**
  - **AnalyzeNewsModule: Reprezintă logica din analyze_news.py (interacțiune AI, parsare, salvare DB).**
  - **AlertMonitorModule: Reprezintă logica din alert_monitor.py (verificare VIX, trimitere alerte).**
  - **AppFlaskModule: Reprezintă logica din appFlask.py (aplicația web, rute, servire date).**
  - **SchedulerModule: Reprezintă logica din scheduler_main.py (programare task-uri).**
- **Entități de Date Principale (din baza de date, pot fi reprezentate ca și clase simple de date):**
  - **SentimentHistoryEntity:**
  - **Atribute: id, timestamp, fear_greed, vix, summary_text.**
  - **VixAlertSubscriptionEntity:**

- **Atribute: id, email, vix_threshold, is_active, created_at, last_alert_sent_at**

1. **Relații:**

- **SchedulerModule utilizează/apelează WebScrapeModule, AnalyzeNewsModule, AlertMonitorModule.**
- **AppFlaskModule utilizează/apelează WebScrapeModule (indirect, prin buton UI), AnalyzeNewsModule (indirect, prin buton UI).**
- **AnalyzeNewsModule scrie în SentimentHistoryEntity (în DB).**
- **AppFlaskModule citește din SentimentHistoryEntity și scrie/citește din VixAlertSubscriptionEntity (în DB).**
- **AlertMonitorModule citește din VixAlertSubscriptionEntity și utilizează (o funcție din) WebScrapeModule pentru a obține VIX.**

2. Diagrama de Secvențe

Această diagramă ilustrează fluxul de interacțiuni între diferite componente pentru un scenariu specific.

- **Scenariul 1: Actualizare Automată a Datelor și Verificare Alerte (Rulat de Scheduler)**
    - **Scheduler (scheduler_main.py) -> WebScrapeModule (webScrape.py): execute_script()**
    - **WebScrapeModule -> ExternalAPIs/RSS: fetch_news_and_vix()**
    - **ExternalAPIs/RSS -->> WebScrapeModule: news_data, vix_data**
    - **WebScrapeModule -> FileSystem: save_to_json("financial_news_agg.json")**
    - **Scheduler -> AnalyzeNewsModule (analyze_news.py): execute_script() (după succesul webScrape)**
    - **AnalyzeNewsModule -> FileSystem: load_from_json("financial_news_agg.json")**
    - **AnalyzeNewsModule -> AzureAIService (DeepSeek): analyze_text(news_articles)**
    - **AzureAIService -->> AnalyzeNewsModule: ai_summary, f_g_index**
    - **AnalyzeNewsModule -> PostgreSQL DB: INSERT INTO sentiment_history(...)**
    - **AnalyzeNewsModule -> FileSystem: save_to_json("latest_indices.json")**

- **Scheduler -> AlertMonitorModule (alert_monitor.py): execute_script()**
- **AlertMonitorModule -> WebScrapeModule (sau funcție yfinance): get_vix_value()**
- **WebScrapeModule -->> AlertMonitorModule: current_vix**
- **AlertMonitorModule -> PostgreSQL DB: SELECT FROM vix_alerts_subscriptions WHERE ...**
- **PostgreSQL DB -->> AlertMonitorModule: subscriptions_to_alert**
- **Pentru fiecare subscription în subscriptions_to_alert:**
- **AlertMonitorModule -> SMTPServer: send_email(...)**
- **AlertMonitorModule -> PostgreSQL DB: UPDATE vix_alerts_subscriptions SET last_alert_sent_at = ...**

3. **Diagrama de Stare (Opțional)**

Prezintă stările prin care poate trece o entitate importantă.

Entitate: Subscriere la Alerte VIX (VixAlertSubscriptionEntity)

Stări:

- **Inexistentă**
- **Activă (Nealertată Recent): Subscrierea este activă, pragul nu este depășit sau a trecut suficient timp de la ultima alertă.**
- **Activă (Alertată Recent): Subscrierea este activă, pragul este depășit, dar o alertă a fost trimisă recent (în intervalul MIN_ALERT_INTERVAL).**
- **Inactivă (dacă ar fi fost implementat sistem de dezabonare).**

Tranziții:

- **Inexistentă -> Activă (Nealertată Recent): Utilizatorul se abonează (INSERT în DB).**
- **Activă (Nealertată Recent) -> Activă (Alertată Recent): VIX depășește pragul, alertă trimisă, last_alert_sent_at actualizat.**
- **Activă (Alertată Recent) -> Activă (Nealertată Recent): Timpul scurs de la last_alert_sent_at depășește MIN_ALERT_INTERVAL (iar VIX este încă peste prag, sau a scăzut și a crescut din nou).**
- **Activă (Nealertată Recent) -> Activă (Nealertată Recent): VIX nu depășește pragul la verificarea alert_monitor.py.**
- **Activă (Alertată Recent) -> Activă (Alertată Recent): VIX este încă peste prag, dar MIN_ALERT_INTERVAL nu a trecut.**

- **Activă (...) -> Activă (...): Utilizatorul actualizează pragul (UPDATE în DB, last_alert_sent_at s-ar putea reseta sau nu, depinde de logica ta).**
- **(Oricare Stare Activă) -> Inactivă: Utilizatorul se dezabonează (UPDATE is_active=FALSE).**
- **Inactivă -> Activă (Nealertată Recent): Utilizatorul se re-abonează (UPDATE is_active=TRUE, actualizează pragul).**

3.4 Tehnologii și stack de dezvoltare

Enumerarea tehnologiilor utilizate (limbaje de programare, baze de date, framework-uri etc.).

Această secțiune enumeră principalele tehnologii, limbaje de programare, baze de date, framework-uri și biblioteci utilizate în dezvoltarea aplicației "Market Sentiment Dashboard & AI Analyzer".

1. **Limbaje de Programare Principale:**
    1. **Python(versiunea 3.x): Utilizat pentru dezvoltarea întregului backend, incluzând aplicația web Flask, scripturile de colectare a datelor (webScrape.py), scriptul de analiză AI (analyze_news.py), scriptul de monitorizare a alertelor (alert_monitor.py) și scheduler-ul (scheduler_main.py).**
    2. **JavaScript (ES6+): Utilizat pentru logica client-side a dashboard-ului, interactivitate și actualizări dinamice (ex: inițializarea graficelor, export PDF).**

3. **CSS3: Folosit pentru stilizarea și designul vizual al interfeței web.**
4. **SQL: Limbajul standard de interogare utilizat pentru interacțiunea cu baza de date PostgreSQL**

2. **Baze de Date și Stocare:**
   1. **SQL: Limbajul standard de interogare utilizat pentru interacțiunea cu baza de date PostgreSQL.**
   2. **psycopg2-binary: Adaptor Python pentru conectarea și interacțiunea cu baza de date PostgreSQL.**

   3. **Fișiere JSON: Utilizate pentru stocarea intermediară a datelor agregate (financial_news_agg.json) și pentru cache-ul celor mai recenți indici (latest_indices.json).**

3. **Colectarea și Parsarea Datelor (Data Scraping & Parsing):**

   1. requests: Bibliotecă Python pentru efectuarea cererilor HTTP către API-uri și surse web.

   2. feedparser: Bibliotecă Python pentru parsarea fluxurilor de știri RSS.

   3. newsapi-python: Client Python oficial pentru interacțiunea cu NewsAPI.org.

   4. yfinance: Bibliotecă Python pentru descărcarea datelor financiare (VIX, știri per ticker) de pe Yahoo Finance.

   5. BeautifulSoup4 (bs4): Bibliotecă Python pentru parsarea documentelor HTML și XML (utilizată pentru curățarea sumarurilor de știri).

   6. html (modul Python standard): Folosit pentru decodarea entităților HTML.

   7. re (modul Python standard): Folosit pentru operațiuni cu expresii regulate (ex: parsarea răspunsului AI).

4. **Backend și Framework-uri:**

    1. **Flask: Micro-framework web Python utilizat pentru construirea aplicației web și a API-urilor interne (ex: pentru rularea scripturilor din UI).**
    2. **Flask-WTF: Extensie Flask pentru lucrul cu formulare web, incluzând validarea datelor și protecția CSRF (utilizată pentru formularul de subscriere la alerte).**
5. **Inteligență Artificială (AI) și Servicii Cloud:**
    1. **Azure AI Services: Platforma cloud Microsoft utilizată pentru a hosta și accesa modelul de limbaj DeepSeek.**
    2. **Model AI: DeepSeek (deployat pe Azure): Modelul lingvistic mare (LLM) utilizat pentru analiza de sentiment a știrilor și generarea indicelui Fear & Greed.**
    3. **azure-ai-inference SDK: Biblioteca Python furnizată de Azure pentru interacțiunea programatică cu endpoint-urile modelelor AI deployate.**
6. **Frontend și Vizualizare Date:**
    1. **Gauge.js: Bibliotecă JavaScript pentru crearea indicatorului vizual de tip "gauge" (pentru F&G Index).**
    2. **Chart.js: Bibliotecă JavaScript pentru generarea graficelor de tip linie interactive (pentru trendurile istorice F&G și VIX).**
    3. **html2canvas: Bibliotecă JavaScript utilizată pentru a captura conținut HTML ca imagine (pentru exportul PDF).**
    4. **jsPDF: Bibliotecă JavaScript utilizată pentru a genera fișiere PDF client-side.**
7. **Automatizare și Programare Task-uri:**
    1. **schedule: Bibliotecă Python simplă utilizată în scheduler_main.py pentru a programa rularea periodică a scripturilor de backend.**
    2. **subprocess (modul Python standard): Utilizat în scheduler_main.py și appFlask.py pentru a rula scripturile Python externe (webScrape.py, analyze_news.py, alert_monitor.py) ca procese separate.**
8. **Trimitere Email-uri:**
    1. **smtplib (modul Python standard): Utilizat în alert_monitor.py pentru a trimite emailuri de alertă prin protocolul SMTP.**
    2. **email.mime.text (modul Python standard): Utilizat pentru a construi conținutul mesajelor email.**
9. **Testare:**
    1. **pytest: Framework de testare utilizat pentru scrierea și rularea testelor unitare și de integrare.**
    2. **pytest-mock (și unittest.mock): Utilizat pentru a simula (mock-ui) dependențe externe (ex: apeluri API, conexiuni la baza de date) în timpul testelor.**

10. **Managementul Mediului și Dependențelor:**
    1. **Python Virtual Environments (venv): Utilizat pentru a izola dependențele proiectului.**
    2. **pip și requirements.txt: Pentru gestionarea pachetelor Python.**

3.5 Interfața utilizatorului (UI/UX)

Descrierea designului interfeței utilizatorului, mockups, wireframes.

Această secțiune descrie designul interfeței utilizatorului pentru aplicația "Market Sentiment Dashboard & AI Analyzer", principiile UI/UX care au ghidat dezvoltarea acesteia și tehnologiile frontend utilizate. Scopul principal al interfeței este de a prezenta informațiile complexe despre sentimentul pieței într-un mod clar, intuitiv și accesibil.

1. **Principii UI/UX aplicate**
    1. **Claritate și Intuitivitate: Designul se concentrează pe prezentarea directă și ușor de înțeles a indicatorilor cheie (F&G Index, VIX) și a sumarului AI. Navigarea este minimală, toate informațiile relevante fiind disponibile pe o singură pagină principală (dashboard).**
    2. **Consistență Vizuală: S-a urmărit o temă vizuală unitară (dark theme "premium") pe toate elementele dashboard-ului, folosind o paletă de**

culori definită și o tipografie consistentă pentru a crea o experiență coerentă.

3. Feedback Utilizator: Aplicația oferă feedback vizual pentru acțiunile utilizatorului, cum ar fi starea de încărcare a butoanelor de rulare a scripturilor și mesaje de confirmare/eroare (prin alert() și mesaje flash) la subscrierea la alerte.
4. Ierarhie Informațională: Informațiile sunt structurate logic în carduri distincte, cu titluri clare, pentru a facilita scanarea rapidă și înțelegerea datelor. Cele mai importante date (F&G, VIX curent) sunt prezentate proeminent.
5. Responsivitate: Interfața este proiectată pentru a fi adaptabilă și funcțională pe diferite dimensiuni de ecran (desktop, tabletă, mobil), prin utilizarea tehnicilor de design responsiv în CSS (media queries, flexbox, grid).
6. Design Modern și Atractiv: S-a optat pentru un design modern, cu o temă întunecată, utilizând spațiere adecvată, umbre subtile și elemente vizuale (gauge, grafice, iconițe) pentru a îmbunătăți atractivitatea și angajamentul utilizatorului.
7. Accesibilitate (Considerații de Bază): Deși nu s-a efectuat o testare exhaustivă a conformității WCAG, s-a încercat utilizarea unui contrast suficient între text și fundal și s-au adăugat atribute alt la imagini. (Poți adăuga mai multe aici dacă ai făcut eforturi specifice).

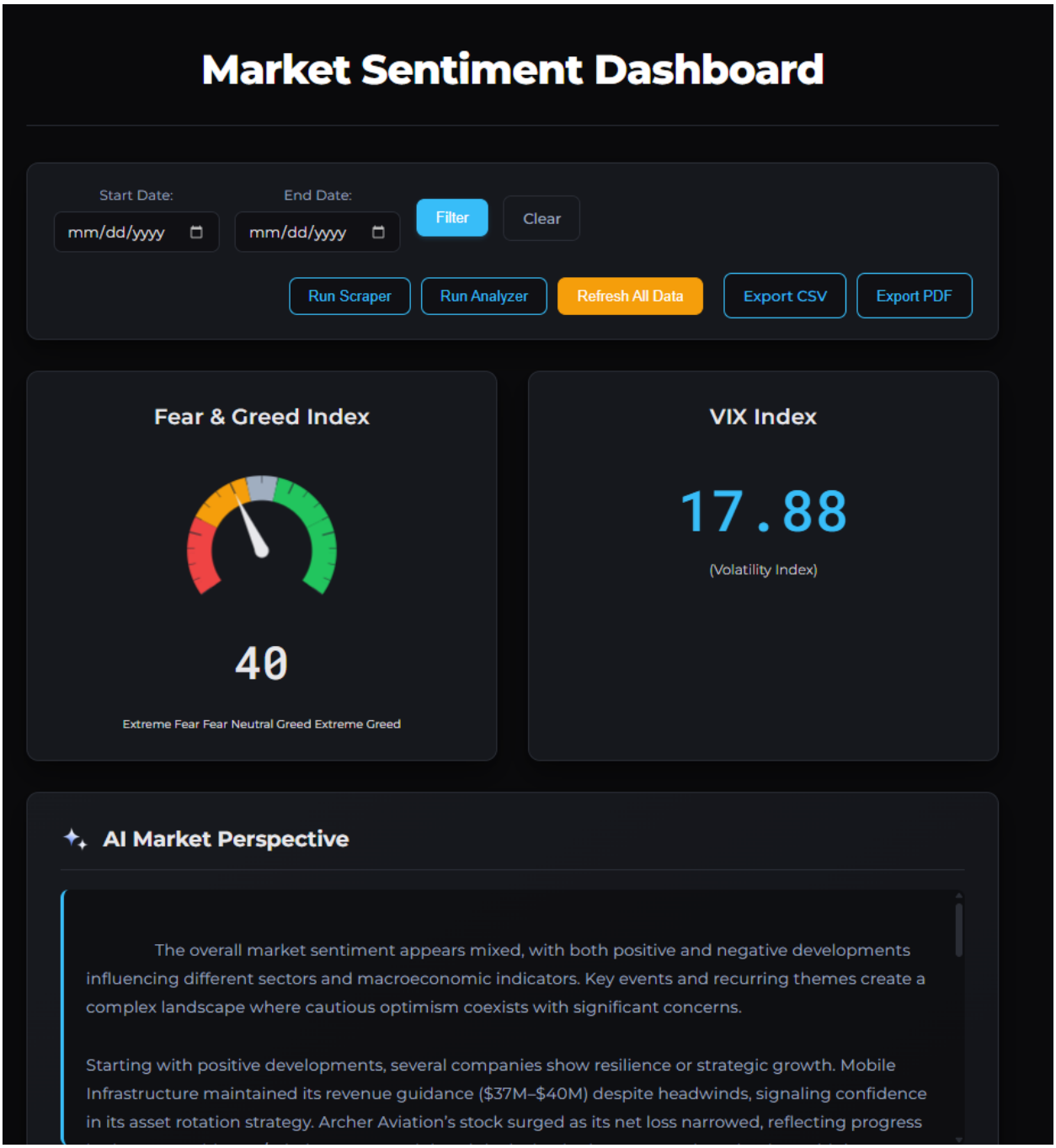
2. Structura Interfeței (Pagina Principală - Dashboard):
    1. Aplicația constă dintr-o singură pagină principală care servește ca dashboard centralizator:
    2. Antet Principal: Titlul aplicației "Market Sentiment Dashboard".
    3. Secțiunea de Controale (controls-row):
    4. Formular de Filtrare pe Dată: Permite utilizatorului să selecteze un interval de dată (start, end) pentru a filtra datele istorice afișate. Include butoane "Filter" și "Clear".
    5. Butoane de Acțiune Pipeline: Butoane ("Run Scraper", "Run Analyzer", "Refresh All Data") pentru declanșarea manuală a scripturilor de backend.
    6. Butoane de Export: Butoane ("Export CSV", "Export PDF") pentru descărcarea datelor sau a vizualizării curente.
- Secțiunea Metricilor Curente (metrics-row):
    1. Card "Fear & Greed Index": Afișează valoarea numerică curentă a indicelui F&G și un indicator vizual de tip gauge. Include etichete

pentru zonele indicelui (Extreme Fear, Fear, Neutral, Greed, Extreme Greed).

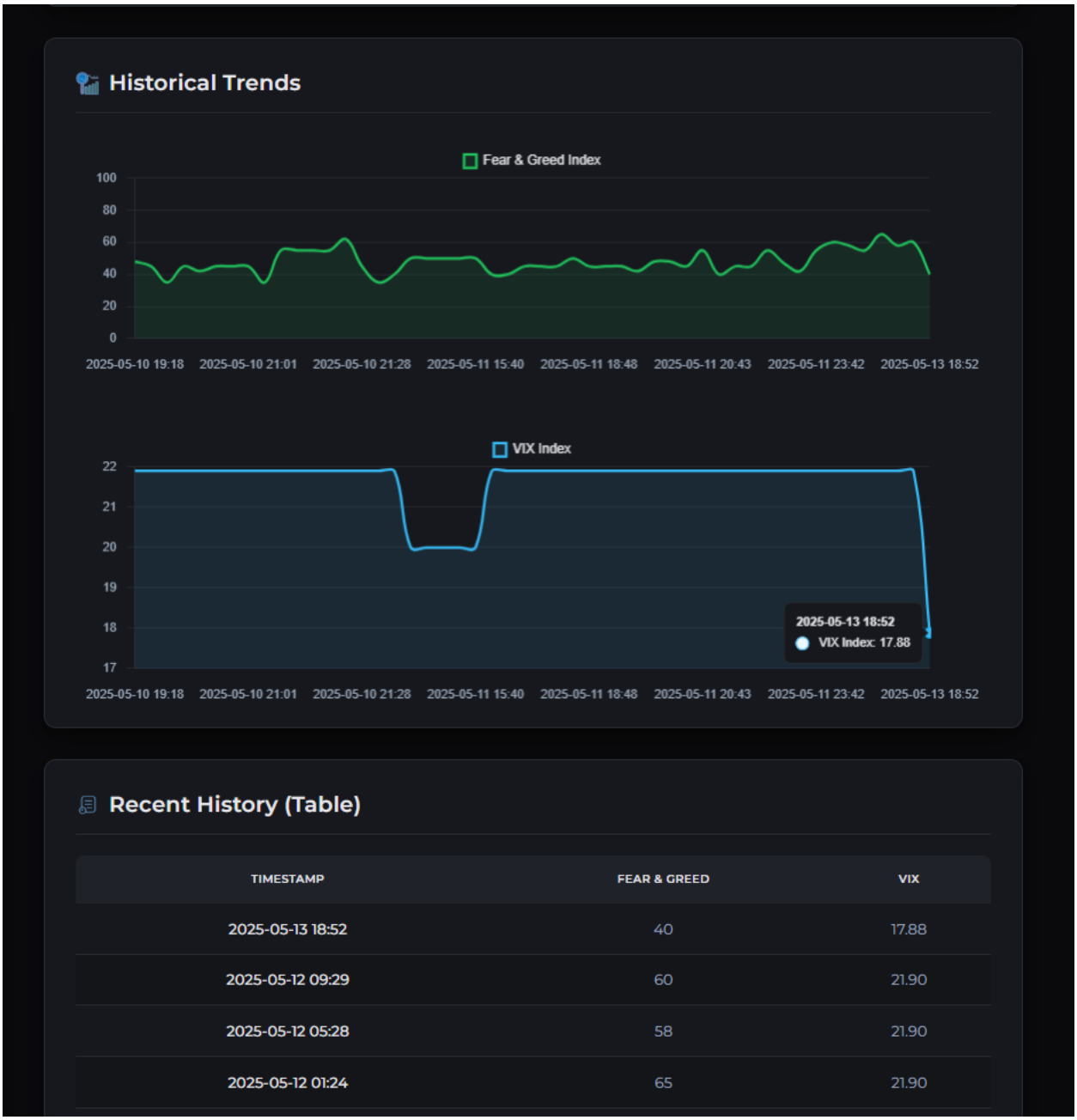
2. **Card "VIX Index": Afișează valoarea numerică curentă a indicelui VIX și o scurtă descriere.**

- **Card "Historical Trends":**
    1. **Conține două grafice de tip linie generate cu Chart.js:**
    2. **Un grafic pentru evoluția istorică a indicelui Fear & Greed.**
    3. **Un grafic pentru evoluția istorică a indicelui VIX.**
    4. **Include o iconiță lângă titlu.**
- **Card "Recent History (Table):**
    1. **Afișează un tabel cu cele mai recente înregistrări de sentiment (Timestamp, Fear & Greed, VIX).**
    2. **Include o iconiță lângă titlu.**
    3. **Afișează un mesaj dacă nu există date pentru perioada selectată.**
- **Card "VIX Alert Subscription":**
    1. **Conține un formular HTML (gestionat cu Flask-WTF) unde utilizatorii își pot introduce adresa de email și un prag VIX dorit pentru a se abona la notificări.**
    2. **Afișează mesaje flash pentru succesul sau eșecul subscrierii.**
    3. **Footer: Afișează data și ora ultimei actualizări a analizei.**

3. **Tehnologii și Biblioteci UI:**
    - **Framework Backend (pentru servire HTML): Flask (Python).**
    - **Template Engine: Jinja2 (integrat în Flask).**
    - **Structură: HTML5.**
    - **Stilizare: CSS3 (cu variabile CSS pentru temă și design responsiv).**
    - **Interactivitate Client-Side: JavaScript (ES6+).**
    - **Biblioteci de Vizualizare Date:**
        1. **Gauge.js: Pentru indicatorul F&G.**
        2. **Chart.js: Pentru graficele istorice.**
    - **Biblioteci pentru Export PDF (Client-Side):**
        1. **html2canvas: Pentru a captura imaginea dashboard-ului.**
        2. **jsPDF: Pentru a genera fișierul PDF din imagine.**
    - **Fonturi: Inter (pentru text general) și Roboto Mono (pentru valori numerice), încărcate prin Google Fonts.**



Figură 2.1 – Card-urile principale



TIMESTAMP	FEAR & GREED	VIX
2025-05-13 18:52	40	17.88
2025-05-12 09:29	60	21.90
2025-05-12 05:28	58	21.90
2025-05-12 01:24	65	21.90

Figură 2.2 – Card-urile cu date istorice

TIMESTAMP	FEAR & GREED	VIX
2025-05-13 18:52	40	17.88
2025-05-12 09:29	60	21.90
2025-05-12 05:28	58	21.90
2025-05-12 01:24	65	21.90
2025-05-12 00:59	55	21.90
2025-05-12 00:33	58	21.90
2025-05-12 00:08	60	21.90
2025-05-11 23:42	55	21.90
2025-05-11 23:17	42	21.90
2025-05-11 22:51	47	21.90
2025-05-11 22:25	55	21.90
2025-05-11 22:00	45	21.90
2025-05-11 21:34	45	21.90
2025-05-11 21:09	40	21.90
2025-05-11 20:43	55	21.90

VIX Alert Subscription

Receive an email notification when the VIX (Volatility Index) goes above your specified threshold.

Your Email Address: your.email@example.com

Alert me when VIX is above: e.g., 30.0

Subscribe / Update Alert

If you are already subscribed, this will update your threshold.

Figură 2.3 – Card-urile cu istoricul intrărilor în baza de date și cel pentru abonament/alerte

4. Implementare

4.1 Structura codului

Descrierea modului în care este organizat codul aplicației.

Codul aplicației "Market Sentiment Dashboard & AI Analyzer" este organizat pe baza unei arhitecturi modulare, urmând principiile separării responsabilităților pentru a facilita dezvoltarea și mentenanța. Structura principală a directoarelor și fișierelor este următoarea:aplicației este organizat pe baza unei arhitecturi modulare, precum NUME ARHITECTURĂ, urmând principiile separării responsabilităților și dezvoltării scalabile.

Structura principală este următoarea:

```
ProiectMarketSentimentMDS/
├── scheduler_main.
├── alert_monitor.py
├── website/
│   ├── __init__.
│   ├── appFlask.py
│   ├── crucialPys/
│   │   ├── __init__.py
│   │   ├── webScrape.py
│   │   ├── analyze_news.py
│   │   └── alert_monitor.py
│   ├── data_files/
│   │   ├── financial_news_agg.json
│   │   └── latest_indices.json
│   ├── static/
│   │   ├── css/
│   │   │   └── style.css
│   │   ├── js/
│   │   │   └── script.js
```

```
│   │   └── images/
│   │       ├── gemini-star.jpg (sau .png)
│   │       ├── market-trends.png
│   │       └── recent-history.png
│   ├── templates/
│   │   └── index.html
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_appFlask.py
│   ├── test_analyze_news.py
│   └── test_webScrape.py
├── scheduler_logs/
│   ├── webscrape_runs.log
│   ├── analyze_news_runs.log
│   └── alert_monitor_runs.log
├── .gitignore
└── README.md
```

1. Nivelul Rădăcină al Proiectului (ProiectMarketSentimentMDS/):
    1. scheduler_main.py: Scriptul principal Python care folosește biblioteca schedule pentru a orchestra rularea periodică a scripturilor de colectare, analiză și monitorizare a alertelor.
    2. scheduler_main.py: Scriptul principal Python care folosește biblioteca schedule pentru a orchestra rularea periodică a scripturilor de colectare, analiză și monitorizare a alertelor.
    3. scheduler_main.py: Scriptul principal Python care folosește biblioteca schedule pentru a orchestra rularea periodică a scripturilor de colectare, analiză și monitorizare a alertelor.
    4. tests/: Conține toate testele automate (unitare, de integrare) scrise cu pytest.

5. tests/: Conține toate testele automate (unitare, de integrare) scrise cu pytest.

2. Directorul Aplicației Web (website/):
    1. appFlask.py: Nucleul aplicației web Flask. Definește rutele, gestionează request-urile HTTP, interacționează cu baza de date pentru a prelua datele de sentiment, procesează formularul de subscriere la alerte și randează template-urile HTML. De asemenea, conține rute pentru a declanșa manual scripturile de backend.
    2. crucialPys/: Un pachet Python care conține logica de bază a backend-ului:
    3. webScrape.py: Responsabil pentru colectarea știrilor din diverse surse (RSS, NewsAPI) și a datelor VIX (folosind yfinance). Salvează datele agregate în financial_news_agg.json.
    4. crucialPys/: Un pachet Python care conține logica de bază a backend-ului:
    5. webScrape.py: Responsabil pentru colectarea știrilor din diverse surse (RSS, NewsAPI) și a datelor VIX (folosind yfinance). Salvează datele agregate în financial_news_agg.json.
    6. data_files/: Stochează fișiere JSON intermediare sau de cache.
    7. static/: Conține resursele statice servite direct de browser:
    8. css/style.css: Stilurile principale pentru aspectul vizual al dashboard-ului.
    9. js/script.js: Logica JavaScript pentru interactivitatea frontend-ului (inițializare Gauge.js, Chart.js, funcționalitate export PDF, trimitere request-uri AJAX pentru rularea scripturilor).
    10. images/: Imagini și iconițe folosite în interfață.
    11. templates/index.html: Template-ul Jinja2 principal care definește structura și conținutul dashboard-ului web.
    12. forms.py (Opțional): Dacă se utilizează Flask-WTF, acest fișier ar conține definițiile claselor de formulare (ex: VixAlertSubscriptionForm). (În implementarea curentă, această clasă este definită în appFlask.py).

Această structură modulară permite o separare clară a responsabilităților între componenta de colectare a datelor, componenta de analiză AI, componenta de alertare, aplicația web și scheduler-ul de task-uri.

4.2 Module și componente

Prezentarea modulelor software și a funcționalității fiecăruia.

Această secțiune descrie principalele module software ale aplicației "Market Sentiment Dashboard & AI Analyzer" și funcționalitatea specifică fiecăruia. Aplicația este organizată într-o manieră modulară pentru a separa responsabilitățile și a facilita înțelegerea și dezvoltarea.

1. **Modulul de Colectare a Datelor (webScrape.py)**
    - **Responsabilități:**
        1. **Preluarea automată a articolelor de știri financiare din multiple surse predefinite:**
            1. **Fluxuri RSS (ex: Yahoo Finance, Investing.com, Google News).**
            2. **NewsAPI (ex: Bloomberg, Reuters, CNBC), folosind interogări specifice și cheia API configurată.**
        2. **Preluarea știrilor specifice pentru o listă predefinită de tickere bursiere de pe Yahoo Finance (folosind yfinance).**
        3. **Preluarea valorii curente/recente a indicelui de volatilitate VIX de pe Yahoo Finance.**
        4. **Curățarea preliminară a conținutului știrilor (eliminarea tag-urilor HTML din sumare).**
        5. **Formatarea timestamp-urilor articolelor într-un format standardizat UTC.**
        6. **Deduplicarea articolelor pe baza URL-ului.**
        7. **Limitarea numărului de articole per sursă și a numărului total de articole.**
        8. **Salvarea datelor agregate (știri și VIX) într-un fișier JSON intermediar (financial_news_agg.json) pentru procesare ulterioară.**
    - **Componente Cheie: Funcții dedicate pentru fiecare tip de sursă de date (get_rss_news, get_newsapi_news, get_yfinance_news), funcții helper pentru curățarea HTML (clean_html_summary) și formatarea datelor (format_timestamp).**

2. **Modulul de Analiză AI și Persistență (analyze_news.py)**
    - **Responsabilități:**
        1. **Încărcarea datelor agregate (știri, VIX) din fișierul JSON generat de webScrape.py.**
        2. **Pregătirea și trimiterea textului articolelor de știri către modelul AI DeepSeek (deployat pe Azure AI Services) pentru analiză de sentiment.**
        3. **Parsarea răspunsului text de la modelul AI pentru a extrage:**
            1. **Un sumar al sentimentului general al pieței.**
            2. **O valoare numerică estimată pentru indicele Fear & Greed (F&G).**
        4. **Salvarea rezultatelor analizei (F&G, VIX, sumarul AI) și a timestamp-ului curent în baza de date PostgreSQL (tabela sentiment_history).**
        5. **Salvarea celor mai recente valori F&G și VIX într-un fișier JSON cache (latest_indices.json) pentru acces rapid.**
    - **Componente Cheie: Funcții pentru interacțiunea cu API-ul Azure AI (analyze_news_with_deepseek), parsarea răspunsului AI (parse_analysis_results), și salvarea datelor în baza de date (save_results_to_db) și în fișierul JSON (save_indices_to_json).**

3. **Modulul Aplicației Web (Flask - appFlask.py)**
    - **Responsabilități:**
        1. **Servirea interfeței web (dashboard-ului) către utilizator.**
        2. **Definirea rutelor HTTP pentru accesarea paginii principale și a funcționalităților de export și control.**
        3. **Preluarea datelor de sentiment (curente și istorice) din baza de date PostgreSQL pentru afișare.**
        4. **Procesarea filtrelor de dată trimise de utilizator și actualizarea datelor afișate.**
        5. **Gestionarea formularului de subscriere la alerte VIX: validarea input-ului și salvarea/actualizarea subscrierilor în baza de date (tabela vix_alerts_subscriptions).**
        6. **Afișarea mesajelor de feedback (flash messages) către utilizator.**
        7. **Generarea și servirea fișierelor CSV pentru exportul datelor istorice.**
        8. **Declanșarea execuției scripturilor de backend (webScrape.py, analyze_news.py) prin intermediul unor rute specifice, la cererea utilizatorului din interfața web.**
    - **Componente Cheie: Instanța aplicației Flask, funcții de rută (@app.route), funcția get_sentiment_data_from_db pentru logica de preluare și procesare a**

datelor din DB, clasa VixAlertSubscriptionForm (Flask-WTF), funcția execute_script_on_server.

4. **Modulul Scheduler (scheduler_main.py)**
    - **Responsabilități:**
        1. **Orchestrarea rulării automate și periodice a scripturilor de backend: webScrape.py, analyze_news.py, și alert_monitor.py.**
        2. **Configurarea intervalelor de timp pentru fiecare task programat.**
        3. **Logarea execuției și a eventualelor erori ale scripturilor rulate într-un director dedicat (scheduler_logs/).**
    - **Componente Cheie: Utilizarea bibliotecii schedule pentru programare, funcția run_script pentru execuția subproceselor și gestionarea logării.**

5. **Componente Frontend (HTML, CSS, JavaScript)**
    - **templates/index.html: Definește structura dashboard-ului, incluzând containerele pentru gauge, grafice, tabel, sumar AI, formularul de filtrare, butoanele de acțiune și formularul de subscriere la alerte. Utilizează Jinja2 pentru inserarea datelor dinamice.**
    - **static/css/style.css: Conține toate regulile de stilizare pentru a crea aspectul vizual "premium" și responsiv al dashboard-ului.**
    - **static/css/style.css: Conține toate regulile de stilizare pentru a crea aspectul vizual "premium" și responsiv al dashboard-ului.**
        1. **static/css/style.css: Conține toate regulile de stilizare pentru a crea aspectul vizual "premium" și responsiv al dashboard-ului.**
        2. **static/css/style.css: Conține toate regulile de stilizare pentru a crea aspectul vizual "premium" și responsiv al dashboard-ului.**
        3. **static/css/style.css: Conține toate regulile de stilizare pentru a crea aspectul vizual "premium" și responsiv al dashboard-ului.**
    - **static/css/style.css: Conține toate regulile de stilizare pentru a crea aspectul vizual "premium" și responsiv al dashboard-ului.**

4.3 Fluxul de date

Descrierea modului în care datele sunt procesate în aplicație.

Această secțiune descrie modul în care datele sunt achiziționate, procesate, stocate și prezentate în cadrul aplicației "Market Sentiment Dashboard & AI Analyzer".

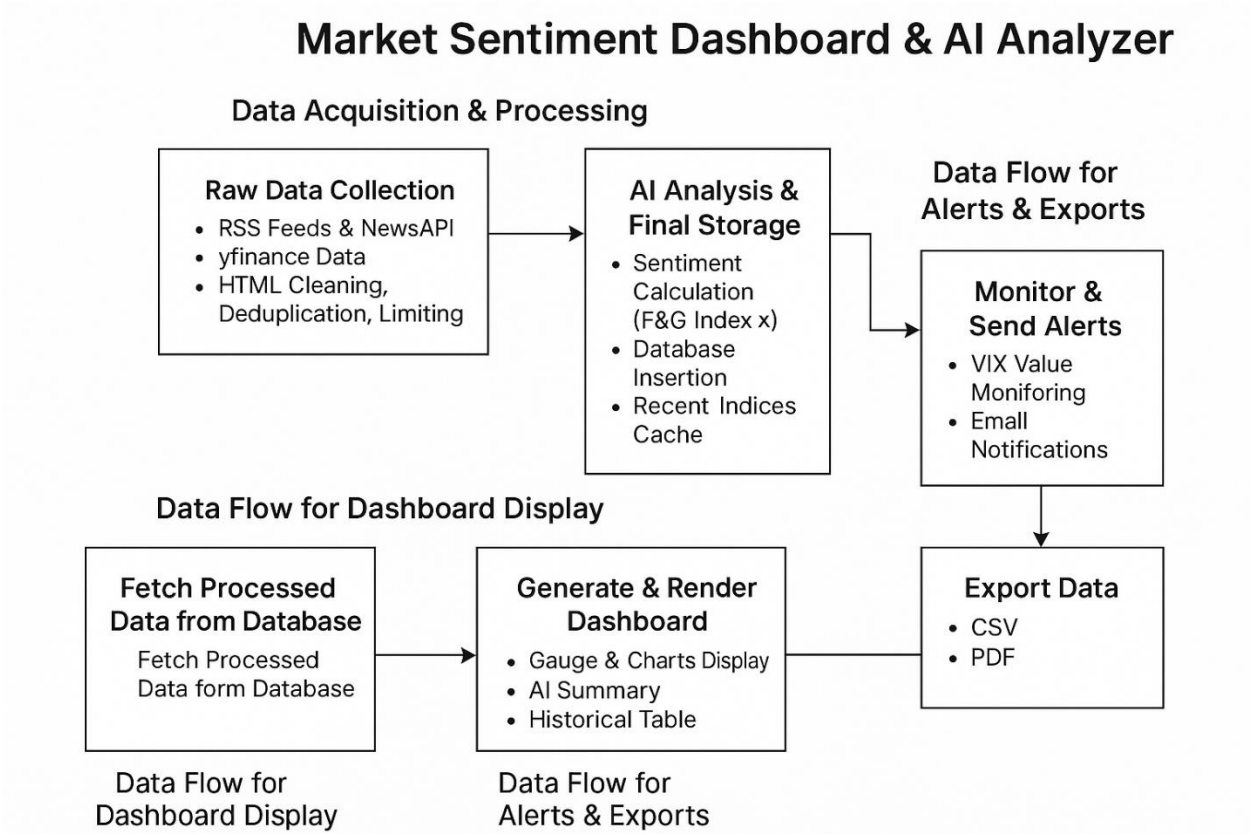
1. Fluxul de Colectare și Analiză a Datelor (Automatizat prin scheduler_main.py sau Rulat Manual):
    - Etapa 1: Colectarea Datelor Brute (webScrape.py)
        1. Inițiere: Scriptul webScrape.py este executat (fie de scheduler, fie manual din UI/linie de comandă).
        2. Preluare Știri RSS: Scriptul iterează prin lista ALL_RSS_FEEDS, face request-uri HTTP către fiecare URL de flux RSS și parsează conținutul XML folosind feedparser pentru a extrage titluri, link-uri, sumare și timestamp-uri.
        3. Preluare Știri NewsAPI: Scriptul interoghează NewsAPI folosind cheia API și parametrii de căutare (NEWSAPI_QUERY, NEWSAPI_SOURCES) pentru a obține articole de știri relevante.
        4. Preluare Date yfinance: Scriptul preia valoarea curentă a indicelui VIX (^VIX) și, opțional, știri pentru alte tickere specificate.
        5. Procesare Primară:
            1. Curățare HTML: Sumarele știrilor sunt curățate de tag-uri HTML folosind BeautifulSoup4 (dacă este disponibil) sau expresii regulate.
            2. Formatare Timestamp: Toate timestamp-urile articolelor sunt convertite într-un format ISO 8601 standardizat, în UTC.
            3. Deduplicare: Articolele sunt deduplicate pe baza URL-ului normalizat pentru a evita procesarea aceleiași știri de mai multe ori.
            4. Limitare: Numărul de articole per sursă și numărul total de articole sunt limitate conform constantelor definite.
        6. Curățare HTML: Sumarele știrilor sunt curățate de tag-uri HTML folosind BeautifulSoup4 (dacă este disponibil) sau expresii regulate.
        7. Formatare Timestamp: Toate timestamp-urile articolelor sunt convertite într-un format ISO 8601 standardizat, în UTC.
        8. Deduplicare: Articolele sunt deduplicate pe baza URL-ului normalizat pentru a evita procesarea aceleiași știri de mai multe ori.
        9. Limitare: Numărul de articole per sursă și numărul total de articole sunt limitate conform constantelor definite.
    - Etapa 2: Analiza AI și Stocarea Finală (analyze_news.py)

1. Inițiere: Scriptul analyze_news.py este executat (după finalizarea cu succes a webScrape.py).
2. Încărcare Date Agregate: Scriptul citește datele din financial_news_agg.json (articolele și valoarea VIX).
3. Interacțiune cu Azure AI (DeepSeek): Textul articolelor de știri este trimis către endpoint-ul modelului DeepSeek pe Azure AI Services.
4. Primire și Parsare Răspuns AI: Scriptul primește răspunsul text de la AI și folosește funcția parse_analysis_results pentru a extrage valoarea numerică a indicelui Fear & Greed (F&G) și sumarul text al sentimentului pieței.
5. Stocare în Baza de Date: Rezultatele (F&G index, valoarea VIX curentă, sumarul AI) împreună cu un timestamp UTC curent sunt inserate ca o nouă înregistrare în tabela sentiment_history din baza de date PostgreSQL, folosind psycopg2.
6. Actualizare Cache Indici Recenți: Cele mai recente valori F&G și VIX sunt salvate în fișierul website/data_files/latest_indices.json pentru acces rapid de către aplicația web.

2. Fluxul de Date pentru Afișarea Dashboard-ului (appFlask.py și Frontend):
    1. Request Utilizator: Utilizatorul accesează URL-ul principal al aplicației (ex: http://127.0.0.1:5000/) în browser. Pot fi incluși parametri de filtrare pe dată în URL (ex: ?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD).
    2. Procesare în Flask (appFlask.py - ruta index()):
        1. Aplicația Flask primește request-ul.
        2. Apelează funcția get_sentiment_data_from_db(), pasând eventualii parametri de filtrare pe dată.
        3. Interogare Bază de Date: get_sentiment_data_from_db() se conectează la PostgreSQL și execută query-uri SQL:
        4. Un SELECT ... ORDER BY timestamp DESC LIMIT 1 pentru a prelua cea mai recentă înregistrare de sentiment (F&G, VIX, sumar AI, timestamp).
        5. Un SELECT ... WHERE [condiții_data] ORDER BY timestamp DESC LIMIT [limit_chart] pentru a prelua datele istorice necesare pentru grafice și tabel, aplicând filtrele de dată dacă sunt specificate.
        6. Procesare Date pentru Afișare: Datele preluate din DB sunt formatate pentru afișare (ex: timestamp-uri, valori numerice). Datele pentru grafice sunt pregătite (inversate pentru a avea cel mai vechi primul).
    3. Randare Template: Aplicația Flask trimite datele procesate către template-ul index.html folosind motorul Jinja2.
    4. Afișare în Browser:

    1. Variabilele JavaScript globale (pentru gauge și grafice) sunt populate cu datele trimise de Flask.
    2. Scriptul static/js/script.js este executat:
    3. Inițializează și populează indicatorul Gauge.js cu valoarea F&G curentă.
    4. Inițializează și desenează graficele Chart.js cu datele istorice F&G și VIX.
    5. Tabelul istoric și sumarul AI sunt afișate direct prin randarea Jinja2.
3. Fluxul de Date pentru Subscrierea la Alerte VIX (Formular Web):
    1. Input Utilizator: Utilizatorul completează formularul de subscriere la alerte VIX din index.html (email și prag VIX) și apasă "Subscribe / Update Alert".
    2. Request POST: Browserul trimite datele formularului printr-un request HTTP POST către ruta / (sau o rută dedicată) a aplicației Flask.
    3. Procesare în Flask (appFlask.py - în ruta index() sau ruta dedicată):
    4. Formularul este validat folosind Flask-WTF.
    5. Dacă este valid, aplicația se conectează la PostgreSQL.
    6. Se verifică dacă emailul există deja în tabela vix_alerts_subscriptions.
    7. Se execută un query INSERT (pentru un email nou) sau UPDATE (pentru un email existent, actualizând pragul și reactivând subscrierea) în tabela vix_alerts_subscriptions.
    8. Se face commit la tranzacție.
    9. Se afișează un mesaj flash de succes sau eroare.
    10. Se face redirect înapoi la pagina principală.
4. Fluxul de Date pentru Monitorizarea și Trimiterea Alertelor VIX (alert_monitor.py):
    1. Inițiere: Scriptul alert_monitor.py este executat periodic de scheduler_main.py.
    2. Preluare VIX Curent: Scriptul apelează get_vix_value_yfinance() pentru a obține cea mai recentă valoare VIX.
    3. Interogare Subscrieri: Se conectează la PostgreSQL și interoghează tabela vix_alerts_subscriptions pentru a găsi subscrierile active al căror prag VIX este depășit de valoarea VIX curentă și pentru care nu s-a trimis o alertă recent (conform MIN_ALERT_INTERVAL).
    4. Trimitere Email: Pentru fiecare subscriere eligibilă:
    5. Se construiește conținutul emailului.
    6. Se stabilește o conexiune la serverul SMTP (folosind smtplib și credentialele din variabilele de mediu).
    7. Se trimite emailul.
    8. Actualizare Bază de Date: Dacă emailul este trimis cu succes, se actualizează câmpul last_alert_sent_at în tabela vix_alerts_subscriptions pentru acea subscriere.

5. Fluxul de Date pentru Exporturi (CSV, PDF):
    1. Utilizatorul apasă "Export CSV".
    2. Browserul face un request GET la ruta /export/csv (cu posibili parametri de filtrare).
    3. Flask (appFlask.py) preia datele relevante din DB (folosind get_sentiment_data_from_db cu for_export=True și filtrele).
    4. Generează conținutul CSV în memorie.
    5. Returnează un Response HTTP care declanșează descărcarea fișierului.
    6. Export PDF:
    7. Utilizatorul apasă "Export PDF".
    8. JavaScript-ul (script.js) din browser folosește html2canvas pentru a captura imaginea dashboard-ului.
    9. jsPDF este folosit pentru a crea un PDF din imagine.
    10. Descărcarea PDF-ului este inițiată client-side. (Serverul nu este implicat direct în generarea PDF-ului).
6. Fluxul de Date pentru Rularea Scripturilor din UI:
    1. Utilizatorul apasă un buton de acțiune (ex: "Refresh All Data").
    2. JavaScript (script.js) face un request AJAX (POST) către ruta Flask corespunzătoare (ex: /run_pipeline).
    3. Flask (appFlask.py):
    4. Primește request-ul.
    5. Folosește modulul subprocess pentru a executa scriptul/scripturile Python relevante (webScrape.py, analyze_news.py) pe server.
    6. Așteaptă finalizarea subprocesului.
    7. Returnează un răspuns JSON către browser indicând succesul sau eșecul.
    8. JavaScript afișează un mesaj utilizatorului și, opțional, reîncarcă pagina.



Figură 3 – Fluxul de Date

5. Testare

5.1 Strategia de testare

Definirea abordării pentru testarea aplicației.

Strategia de testare pentru aplicația "Market Sentiment Dashboard & AI Analyzer" se bazează pe o abordare combinată, incluzând testare manuală și testare automată, pentru a asigura calitatea, corectitudinea funcțională, securitatea de bază și performanța adecvată a software-ului. Procesul de testare se va desfășura în mai multe etape și va include următoarele aspecte principale:

1. Testare Unitară (Automată):
    - Obiectiv: Verificarea funcționării corecte și izolate a fiecărei funcții și componente logice individuale din modulele Python (webScrape.py, analyze_news.py, appFlask.py, alert_monitor.py).
    - METODOLOGIE: Se vor scrie teste unitare pentru funcții cheie, cum ar fi:
        1. Parsarea și curățarea datelor din surse externe (ex: clean_html_summary, format_timestamp în webScrape.py).
        2. Parsarea răspunsului de la modelul AI (ex: parse_analysis_results în analyze_news.py).
        3. Logica de interacțiune cu baza de date (ex: save_results_to_db în analyze_news.py, funcțiile de CRUD pentru vix_alerts_subscriptions în appFlask.py), folosind obiecte simulate (mocks) pentru conexiunea la baza de date.
        4. Logica de procesare a datelor în appFlask.py (ex: get_sentiment_data_from_db), simulând răspunsurile de la baza de date.
        5. Declanșarea scripturilor de backend din UI și verificarea actualizării datelor.
        6. Se utilizează biblioteca pytest-mock (și implicit unittest.mock) pentru a simula dependențele externe (ex: apeluri API, conexiuni la baza de date, funcții ale sistemului de fișiere) și a izola unitatea testată.

    - Instrumente: Framework-ul pytest împreună cu pytest-mock (pentru unittest.mock) pentru scrierea și rularea testelor automate și pentru simularea dependențelor externe.
2. Testare de Integrare (Automată și Manuală):
    - Obiectiv: Asigurarea faptului că diferitele module și componente ale aplicației funcționează corect împreună.
        1. Funcționarea scheduler-ului scheduler_main.py și rularea corectă a scripturilor programate (verificând log-urile și actualizarea datelor).

3. Testare de Performanță
    - Obiectiv: Evaluarea timpului de răspuns al aplicației web și al scripturilor de backend în condiții normale de utilizare.
    - Metodologie: Deși nu se vor folosi unelte dedicate de load testing (precum Apache JMeter) pentru acest proiect de laborator, se va monitoriza manual timpul de încărcare al paginii și timpul de execuție al script Testarea rutelor aplicației Flask (appFlask.py) pentru a verifica dacă returnează codurile de status HTTP corecte și dacă datele sunt randate corespunzător în template-uri (cu mock-uirea apelurilor la baza de date).
        1. Testarea endpoint-urilor API create pentru declanșarea scripturilor (/run_webscrape, /run_analyze_news, /run_pipeline) și pentru subscrierea la alerte, verificând răspunsurile JSON.
4. Testare de Securitate (Considerații de Bază):
    - Obiectiv: Identificarea și mitigarea unor vulnerabilități de securitate elementare.
    - METOLODIGIE:
        1. Verificarea faptului că toate cheile API și credentialele sunt gestionate prin variabile de mediu și nu sunt expuse în cod.
        2. Asigurarea că interogările SQL folosesc parametrizare pentru a preveni SQL Injection.
        3. Verificarea utilizării protecției CSRF de către Flask- apoi rularea analyze_news.py și verificarea datelor în latest_indices.json și în baza de date PostgreSQL.
        4. Verificarea interacțiunii dintre scheduler_main.py și scripturile pe care le rulează (prin observarea log-urilor și a rezultatelor).
        5. Verificarea interacțiunii dintre alert_monitor.py, baza de date (pentru subscrieri) și serviciul SMTP (prin verificarea primirii emailurilor de test).
5. Automatizarea Testelor (parțială):
    - Testele unitare și o parte din testele de integrare (endpoint-uri Flask) vor fi automatizate folosind pytest.
    - Aceste teste pot fi rulate frecvent în timpul dezvoltării pentru a asigura regresia și validarea continuă.
    - (Opțional) Aceste teste se pot integra într-un pipeline CI/CD simplu folosind GitHub Actions.

5.2 Tipuri de teste

Pe parcursul dezvoltării aplicației "Market Sentiment Dashboard & AI Analyzer", s-au aplicat sau se vor aplica următoarele tipuri de teste, utilizând instrumente și tehnici specific.

1. Teste Unitare:
    1. Scop: Verifică funcționarea corectă și izolată a celor mai mici unități de cod (funcții individuale, metode).
    2. Exemple Specifice Proiectului:
        1. Testarea funcției clean_html_summary() din webScrape.py cu diverse input-uri HTML.
        2. Testarea funcției format_timestamp() din webScrape.py cu diferite prin scheduler (din log-uri).
        3. Observarea consumului de resurse (CPU, memorie) în timpul rulării scripturilor (la nivel de bază, folosind Task Manager/htop).
2. Testare de Securitate (Minimală/Conceptuală):
    1. Obiectiv: Buna functionare a masurilor de precautie
        1. Testarea funcției parse_analysis_results() din analyze_news.py pentru a extrage corect F&G și sumarul din diverse formate de răspuns AI.
        2. Testarea funcțiilor de conectare la baza de date și de salvare/preluare a datelor (ex: save_results_to_db, get_sentiment_data_from_db), folosind obiecte simulate (mocks) pentru dependențele externe (DB, API-uri).
        3. Instrumente: pytest, pytest-mock.
3. Teste de Integrare:
    1. Scop: Asigură buna funcționare a modulelor interconectate și a interfețelor dintre ele.
    2. Exemple Specifice Proiectului:
        1. Testarea endpoint-urilor definite în appFlask.py (ex: ruta /, /export/csv, rutele pentru rularea scripturilor) pentru a verifica dacă returnificarea și atenuarea riscurilor de securitate de bază.
    3. Abordare:
        1. Verificarea faptului că toate cheile API și credentialele sunt gestionate prin variabile de mediu și nu sunt hardcodate.
        2. Asigurarea utilizării interogărilor SQL parametrizate pentru a preveni SQL Injection.
        3. Utilizarea Flask-WTF pentru protecție CSRF la formularul de subscriere.

4. Automatizarea Testelor (Parțială):
    1. Testele unitare și o parte din testele de integrare (cele pentru rutele Flask) sunt automatizate folosind pytest.
    2. Aceste teste pot fi rulate frecvent în timpul dezvoltării pentru a asigura regresia și validează răspunsurile corecte și interacționează corespunzător cu logica de business și baza de date (simulată). Se folosește clientul de testare Flask.
        1. Testarea fluxului de date între webScrape.py (care produce financial_news_agg.json) și analyze_news.py (care consumă acest fișier).
        2. Testarea interacțiunii dintre analyze_news.py și baza de date pentru salvarea corectă a datelor.
        3. Testarea interacțiunii dintre alert_monitor.py și baza de date (tabela vix_alerts_subscriptions) și cu funcția de trimitere email (simulată).
    3. Instrumente: pytest (cu clientul de testare Flask și pytest-mock), testare manuală a interfețelor.

5. Teste de Utilizabilitate (Informale):
    1. Scop: Evaluarea cât de ușor și intuitiv este de utilizat dashboard-ul.
    2. Metodologie: Navigarea prin interfață, utilizarea tuturor funcționalităților din perspectiva unui utilizator nou pentru a identifica posibile confuzii sau dificultăți.
    3. Instrumente: Observație directă.

5.3 Plan de testare

Planificarea testelor cu scenarii specifice.

Această secțiune definește planul de testare pentru aplicația "Market Sentiment Dashboard & AI Analyzer", incluzând obiectivele testării, scenarii de testare specifice pentru funcționalitățile cheie și instrumentele utilizate.

1. Obiectivele Testării
    1. Corectitudine Funcțională: Asigurarea că toate funcționalitățile implementate (colectare date, analiză AI, afișare dashboard, filtre, exporturi, alerte VIX, rulare scripturi din UI) funcționează conform specificațiilor.
    2. Robustețe și Gestionarea Erorilor: Verificarea comportamentului aplicației în caz de date de intrare incorecte, erori de la API-uri externe sau probleme de conexiune la baza de date.
    3. Utilizabilitate: Evaluarea intuitivității și ușurinței de utilizare a interfeței web.
    4. Performanță (Observațională): Verificarea timpilor de răspuns acceptabili pentru încărcarea paginii și execuția scripturilor de backend.
    5. Securitate (De Bază): Asigurarea implementării măsurilor de securitate elementare (ex: protecția credentialelor, prevenirea SQL injection de bază).

2. Scenarii de Testare
    1. Testarea Colectării Datelor (webScrape.py)
        1. Scenariu TS_WS_01: Rularea scriptului webScrape.py cu acces la toate sursele de date (RSS, NewsAPI, yfinance).
        2. Asigurati variabilele de mediu necesare (ex: NEWSAPI_KEY).
        3. Executati python website/crucialPys/webScrape.py.
        4. Rezultat Așteptat: Scriptul se finalizează fără erori critice. Fișierul website/data_files/financial_news_agg.json este creat/actualizat și conține articole de știri din diverse surse și valoarea VIX. Log-urile din consolă indică succesul preluării de la fiecare sursă sau avertismente/erori gestionate.

2. Testarea Analizei AI și Persistenței (analyze_news.py)
    1. Scenariu TS_AN_01: Rularea scriptului analyze_news.py cu un fișier financial_news_agg.json valid.

    2. Asigurati existența unui financial_news_agg.json valid (generat de webScrape.py).
    3. Asigurati variabilele de mediu pentru Azure AI și DB.
    4. Executati python website/crucialPys/analyze_news.py.
    5. Rezultat Așteptat: Scriptul se finalizează fără erori. Datele (F&G, VIX, sumar) sunt inserate corect în tabela sentiment_history din PostgreSQL. Fișierul latest_indices.json este actualizat. Log-urile indică succesul apelului AI și al operațiunilor DB.

3. Testarea Sistemului de Alerte VIX (alert_monitor.py)
    1. Asigură o subscriere activă în DB cu un prag VIX mic.
    2. Asigură că last_alert_sent_at este NULL sau mai vechi de MIN_ALERT_INTERVAL.
    3. Rulează alert_monitor.py (manual sau prin scheduler) într-un moment când VIX-ul real (sau simulat) este peste prag.
    4. Rezultat Așteptat: Emailul de alertă este primit la adresa specificată. last_alert_sent_at este actualizat în DB. Log-urile indică succes.
    5. Scenariu TS_AM_02: Rularea alert_monitor.py când VIX este peste prag, dar o alertă a fost trimisă recent.
    6. Pași: Similar cu TS_AM_01, dar last_alert_sent_at este recent (mai nou decât MIN_ALERT_INTERVAL).
    7. Rezultat Așteptat: Nu se trimite email. Log-urile indică faptul că utilizatorul a fost omis din cauza intervalului.
    8. Scenariu TS_AM_03: Rularea alert_monitor.py când VIX este sub pragul tuturor utilizatorilor.
    9. Rezultat Așteptat: Nu se trimit emailuri. Log-urile indică "No users to alert".

4. Testarea Scheduler-ului (scheduler_main.py)
    1. Scenariu TS_SCH_01: Rularea scheduler_main.py și monitorizarea pe o perioadă scurtă.
    2. Pași: Rulează python scheduler_main.py (poate cu intervale de testare mai mici).
    3. Rezultat Așteptat: Scripturile (webScrape.py, analyze_news.py, alert_monitor.py) sunt declanșate la intervalele programate. Log-urile din scheduler_logs/ sunt create și conțin output-ul execuției scripturilor. Datele de pe dashboard se actualizează (după refresh).

5. Instrumente Utilizate pentru Testare
    1. pytest: Framework principal pentru scrierea și rularea testelor unitare și a testelor de integrare pentru endpoint-urile Flask.
    2. pytest-mock (bazat pe unittest.mock): Pentru simularea (mock-uirea) dependențelor externe (ex: apeluri API, conexiuni la baza de date, funcții ale sistemului de fișiere) în testele automate.
    3. Clientul de Testare Flask: Furnizat de Flask (și integrat cu pytest prin fixtures) pentru a simula request-uri HTTP către aplicația web fără a rula un server web complet.
    4. Unelte de Dezvoltare ale Browserului (ex: Chrome DevTools, Firefox Developer Tools): Pentru inspecția elementelor HTML/CSS, depanarea JavaScript-ului client-side, monitorizarea request-urilor de rețea și verificarea consolei browserului.
    5. Interfețe Grafice pentru Baza de Date (ex: pgAdmin, DBeaver) sau psql: Pentru verificarea directă a datelor în baza de date PostgreSQL și pentru rularea de comenzi SQL de setup/cleanup pentru teste.
    6. Logare (modulele print și, ideal, logging din Python): Pentru a urmări execuția scripturilor de backend și a diagnostica probleme.

6. Deployment și mentenanță

6.1 Strategie de deployment

Descrierea procesului de implementare a aplicației (CI/CD, servere, containere).

Procesul de implementare (deployment) al aplicației "Market Sentiment Dashboard & AI Analyzer" este conceput în prezent pentru rulare într-un mediu de dezvoltare local. Totuși, arhitectura modulară permite adaptarea pentru medii de testare și producție, urmând principii DevOps pentru o potențială livrare eficientă.ul de implementare al aplicației utilizează o abordare modernă bazată pe DevOps, asigurând livrarea rapidă și sigură a noilor versiuni.

1. Medii de Deploy (Conceptuale pentru Proiectul Actual):
    1. Mediu de Dezvoltare (Development):
        1. Acesta este mediul principal utilizat. Aplicația Flask rulează cu serverul de dezvoltare (app.run(debug=True)). Scripturile de backend (webScrape.py, analyze_news.py, alert_monitor.py) și scheduler-ul (scheduler_main.py) sunt rulate manual sau prin scheduler-ul Python din terminal, pe mașina locală de dezvoltare. Baza de date PostgreSQL este, de asemenea, hostată local sau accesibilă local. Testele unitare și de integrare sunt rulate în acest mediu.
        2. Mediu de Testare (Staging - Ipotetic): Într-un scenariu mai complex, ar exista un mediu de staging (similar celui de producție, dar nu accesibil publicului larg) unde s-ar face validarea finală și testele de acceptanță înainte de un eventual deploy în producție.
        3. Mediu de Testare (Staging - Ipotetic): Într-un scenariu mai complex, ar exista un mediu de staging (similar celui de producție, dar nu accesibil publicului larg) unde s-ar face validarea finală și testele de acceptanță înainte de un eventual deploy în producție.

2. Tehnologii Utilizate/Considerate pentru Deployment și Operare:
    1. Sisteme de Versionare: GitHub (pentru repository-ul de cod, link: https://github.com/dbogdanm/MarketSentiment).
    2. Mediu Virtual Python (venv): Pentru izolarea dependențelor pe mașina de dezvoltare și pe server.
    3. Server de Dezvoltare Flask: Utilizat pentru rularea locală a aplicației web.
    4. scheduler_main.py cu biblioteca schedule: Pentru automatizarea locală a rulării scripturilor de backend.
    5. (Opțional/Viitor) Docker: Pentru containerizarea aplicației.
    6. (Opțional/Viitor) Gunicorn/Waitress: Ca servere WSGI pentru a rula aplicația Flask într-un mediu similar celui de producție.
    7. (Opțional/Viitor) Nginx: Ca reverse proxy în fața serverului WSGI.
    8. (Opțional/Viitor) GitHub Actions: Pentru automatizarea testelor și, potențial, a deployment-ului sau a rulării pipeline-ului de date.
    9. Variabile de Mediu: Pentru gestionarea configurațiilor sensibile (chei API, credentiale DB).
    10. Logare: Prin print-uri în consolă și salvarea output-ului scripturilor în fișiere de log (gestionat de scheduler_main.py și appFlask.py pentru execute_script_on_server).

6.2 Administrarea și monitorizarea

Instrumente utilizate pentru monitorizarea aplicației și a performanței (opțional)

Această secțiune descrie strategiile și instrumentele (predominant de bază) utilizate pentru administrarea și monitorizarea aplicației "Market Sentiment Dashboard & AI Analyzer", cu scopul de a asigura o funcționare cât mai corectă și de a identifica eventuale probleme.

1. Monitorizarea Performanței (Observațională și prin Logare):
    1. Log-urile de Execuție ale Scripturilor: Scriptul scheduler_main.py și funcția execute_script_on_server din appFlask.py generează fișiere de log (în directorul scheduler_logs/ sau prin output în consolă) care înregistrează:
        1. Timestamp-ul de start și final al fiecărui script (webScrape.py, analyze_news.py, alert_monitor.py).
        2. Output-ul standard (stdout) și erorile standard (stderr) ale scripturilor.
        3. Aceste log-uri permit o evaluare post-factum a timpilor de execuție și identificarea scripturilor care rulează lent sau generează erori.
    2. Monitorizarea Resurselor Sistemului (Manuală):
        1. În timpul rulării locale, se poate utiliza Task Manager (Windows) sau top/htop (Linux) pentru a observa consumul de CPU și memorie al proceselor Python, în special în timpul execuției scripturilor de colectare și analiză AI.
    3. Timpul de Răspuns al Aplicației Web (Manual): Timpul de încărcare al dashboard-ului și răspunsul la acțiunile utilizatorului (filtrare, declanșare scripturi) sunt monitorizate manual în browser folosind uneltele de dezvoltare (tab-ul Network).

2. Monitorizarea Erorilor și Stabilității:
    1. Logarea Erorilor: Toate scripturile principale (webScrape.py, analyze_news.py, alert_monitor.py, appFlask.py, scheduler_main.py) includ blocuri try-except pentru a prinde excepțiile și a le loga (prin print() în consolă sau în fișierele de log). Aceasta ajută la diagnosticarea problemelor.
    2. Mesaje de Feedback în UI: Aplicația Flask folosește mesaje flash pentru a informa utilizatorul despre succesul sau eșecul operațiunilor (ex: subscrierea la alerte). JavaScript-ul folosește alert() pentru feedback la rularea scripturilor.

    3. Verificarea Fișierelor de Date: Se poate verifica manual existența și conținutul fișierelor JSON generate (financial_news_agg.json, latest_indices.json) pentru a se asigura că pipeline-ul de date funcționează corect.
    4. Verificarea Bazei de Date: Se poate interoga manual baza de date PostgreSQL pentru a verifica dacă datele sunt inserate și actualizate corect.
3. Administrarea Aplicației (Simplificată):
    1. Configurare prin Variabile de Mediu: Cheile API, credentialele DB și SMTP, și alte setări sunt gestionate prin variabile de mediu, permițând configurarea fără modificarea codului.
    2. Controlul Rulării Scripturilor din UI: Dashboard-ul oferă butoane pentru a declanșa manual scripturile de colectare și analiză, oferind un nivel de control administrativ asupra actualizării datelor.
    3. Scheduler Local (scheduler_main.py): Permite automatizarea rulării scripturilor. Administrarea acestuia implică pornirea și oprirea manuală a scriptului scheduler_main.py și monitorizarea output-ului său în consolă și a fișierelor de log.
    4. Managementul Subscrierilor la Alerte: Utilizatorii își pot crea și actualiza subscrierile prin formularul web. (Managementul avansat, cum ar fi ștergerea subscrierilor de către un admin, nu este implementat).
4. Notificări și Alerte:
    1. Alerte VIX prin Email: Sistemul trimite automat notificări prin email utilizatorilor abonați atunci când indicele VIX depășește pragul setat de aceștia. Aceasta este principala formă de notificare proactivă implementată.
    2. Log-uri ca Formă de Alertare (pentru Dezvoltator): Erorile critice sau eșecurile scripturilor sunt logate și pot fi consultate de dezvoltator pentru intervenție.

6.3 Plan de mentenanță

Actualizări, corecturi de erori, suport tehnic.

Planul de mentenanță pentru aplicația "Market Sentiment Dashboard & AI Analyzer" vizează asigurarea funcționării optime pe termen lung, actualizarea la noile cerințe sau tehnologii și corectarea eventualelor erori. Acesta include intervenții proactive și reactive.

1. Tipuri de Mentenanță:
    1. Mentenanță Corectivă: Identificare: Erori raportate (ipotetic) de utilizatori, erori descoperite în timpul testării continue, sau probleme identificate prin analiza log-urilor generate de scripturi (scheduler_logs/, log-urile consolei Flask).
    2. Acțiune: Depanarea și corectarea bug-urilor din codul Python (backend, scripturi) sau din componentele frontend (HTML, CSS, JavaScript).
2. Mentenanță Preventivă:
    1. Identificare: Monitorizarea performanței API-urilor externe (NewsAPI, Azure AI, yfinance) și a bibliotecilor utilizate pentru a anticipa probleme de compatibilitate sau depreciere.
    2. Acțiune: Actualizarea periodică a dependențelor Python (din requirements.txt) la versiuni stabile noi, după testare. Revizuirea periodică a codului pentru identificarea zonelor ce pot fi optimizate sau refactorizate pentru claritate. Verificarea și actualizarea variabilelor de mediu (ex: reînnoirea cheilor API dacă expiră).
3. Mentenanță Evolutivă:
    1. Identificare: Noi cerințe funcționale sau non-funcționale (ex: adăugarea de noi surse de știri, noi tipuri de analize AI, îmbunătățirea UI/UX pe baza feedback-ului).
    2. Acțiune: Dezvoltarea și integrarea noilor funcționalități în aplicație, urmată de testare corespunzătoare. (Exemple: funcționalitățile de export PDF/CSV, alerte VIX, butoane de control UI adăugate recent).
4. Mentenanță Adaptivă:
    1. Identificare: Schimbări în API-urile externe (ex: NewsAPI își modifică endpoint-urile sau formatul răspunsului), schimbări în versiunile majore ale tehnologiilor de bază (Python, Flask, PostgreSQL, biblioteci JS) care necesită ajustări de compatibilitate.
    2. Acțiune: Modificarea codului pentru a se adapta la noile cerințe ale serviciilor externe sau ale platformelor. Refactorizarea codului pentru a utiliza noi caracteristici ale limbajelor/bibliotecilor.

5. Instrumente și Procese de Mentenanță (Adaptate pentru Proiectul Actual):
    1. Controlul Versiunilor (Git & GitHub): Toate modificările de cod (corecții, noi funcționalități) sunt gestionate prin Git, cu commit-uri descriptive. Branch-urile pot fi folosite pentru dezvoltarea izolată a modificărilor majore.
    2. Backup Bază de Date (Manual/Conceptual): Pentru un proiect de laborator, backup-ul bazei de date PostgreSQL se poate face manual folosind uneltele PostgreSQL (ex: pg_dump). Într-un mediu de producție, s-ar implementa backup-uri automate.
    3. Logare: Fișierele de log generate de scheduler_main.py și output-ul consolei de la appFlask.py (în mod debug) sunt principalele surse pentru identificarea problemelor operaționale și a erorilor din scripturile de backend.
6. Suport Tehnic (Conceptual):
    1. Documentație: README.md și acest document de proiectare oferă informații despre setup, utilizare și arhitectură.
    2. Contact Dezvoltator: Adresa de email a dezvoltatorului este disponibilă pentru întrebări sau probleme (așa cum este specificat în README.md).
    3. <u>Notă: Nu există un sistem formal de suport 24/7 sau canale de comunicare dedicate (Slack, Teams) pentru acest proiect de laborator.</u>

7. Concluzii

Sumarizarea concluziilor principale și recomandări pentru dezvoltări viitoare.

Această secțiune oferă o sinteză a principalelor constatări rezultate din procesul de analiză, proiectare și implementare a aplicației "Market Sentiment Dashboard & AI Analyzer", precum și recomandări concrete pentru dezvoltări și îmbunătățiri viitoare.

1. Rezumat al Principalelor Constatări și Realizări

- **Arhitectură Modulară și Funcțională: Aplicația a fost proiectată și implementată utilizând o arhitectură pe niveluri, cu o separare clară a responsabilităților între colectarea datelor (webScrape.py), analiza AI și persistența (analyze_news.py), monitorizarea alertelor (alert_monitor.py), orchestrarea task-urilor (scheduler_main.py) și prezentarea web (Flask - appFlask.py). Această structură a facilitat dezvoltarea iterativă și permite extinderi viitoare.**
- **Integrare AI pentru Analiză de Sentiment: S-a implementat cu succes integrarea cu un model de inteligență artificială avansat (DeepSeek, prin Azure AI Services) pentru a genera un sumar al sentimentului pieței și un indice numeric personalizat "Fear & Greed", adăugând o dimensiune inteligentă analizei datelor.**
- **Sistem Automat de Alerte VIX: Implementarea unui sistem de subscriere și notificare prin email pentru depășirea unor praguri VIX definite de utilizator adaugă o componentă proactivă și utilă aplicației.**
- **Utilizarea Tehnologiilor Moderne: Proiectul a demonstrat utilizarea unui stack tehnologic relevant, incluzând Python, Flask, PostgreSQL, JavaScript, și servicii cloud AI, alături de bune practici precum gestionarea configurației prin variabile de mediu și testarea unitară/de integrare (cu pytest).**
- **Gestionarea Robustă a Datelor și Erorilor: S-au implementat mecanisme pentru curățarea și formatarea datelor, gestionarea erorilor la apelurile API și interacțiunile cu baza de date, și logarea de bază a operațiunilor.**
- **Funcționalități Extinse pentru Utilizator: Au fost adăugate funcționalități valoroase, precum filtrarea datelor pe intervale de timp, exportul datelor în format CSV și PDF, și posibilitatea de a declanșa manual**

2. Recomandări pentru dezvoltări viitoare

Pe baza experienței acumulate în dezvoltarea acestei versiuni și a potențialului aplicației, se recomandă următoarele direcții pentru îmbunătățiri viitoare:

- Îmbunătățirea Performanței Colectării Datelor:
  - Explorarea utilizării asyncio și aiohttp în webScrape.py pentru a efectua request-urile către multiple surse RSS și API-uri în mod concurent, reducând timpul total de colectare.
- Rafinarea Analizei AI și a Prezentării Sumarului:
  - Experimentarea cu prompt engineering avansat pentru a obține sumare AI și mai nuanțate sau structurate.
  - Considerarea unei interfețe de tip chat (așa cum am discutat) pentru ca utilizatorii să poată int Extinderea Funcționalităților de Alertare:eroga AI-ul despre sumar sau despre date specifice.
- Extinderea Funcționalităților de Alertare:
  - Implementarea unui mecanism de dezabonare facil pentru alertele VIX (ex: link în email).
  - Adăugarea de noi tipuri de alerte (ex: pentru variații mari ale indicelui F&G, pentru știri despre anumite companii/sectoare).
- Îmbunătățirea Experienței Utilizatorului (UX):
  - Înlocuirea alert()-urilor JavaScript cu notificări "toast" mai elegante și mai puțin intrusive pentru feedback-ul acțiunilor din UI.
  - Adăugarea de opțiuni de personalizare a graficelor (ex: selectarea perioadelor predefinite "1 lună", "3 luni", "1 an" pe lângă date picker).
- Automatizarea Proceselor de Deploy și Mentenanță (CI/CD):
  - Crearea unui Dockerfile pentru containerizarea aplicației Flask și a scripturilor de backend.
  - Configurarea unui pipeline CI/CD simplu (ex: folosind GitHub Actions) pentru rularea automată a testelor la fiecare push și, eventual, pentru deployment automat într-un mediu de staging sau producție.